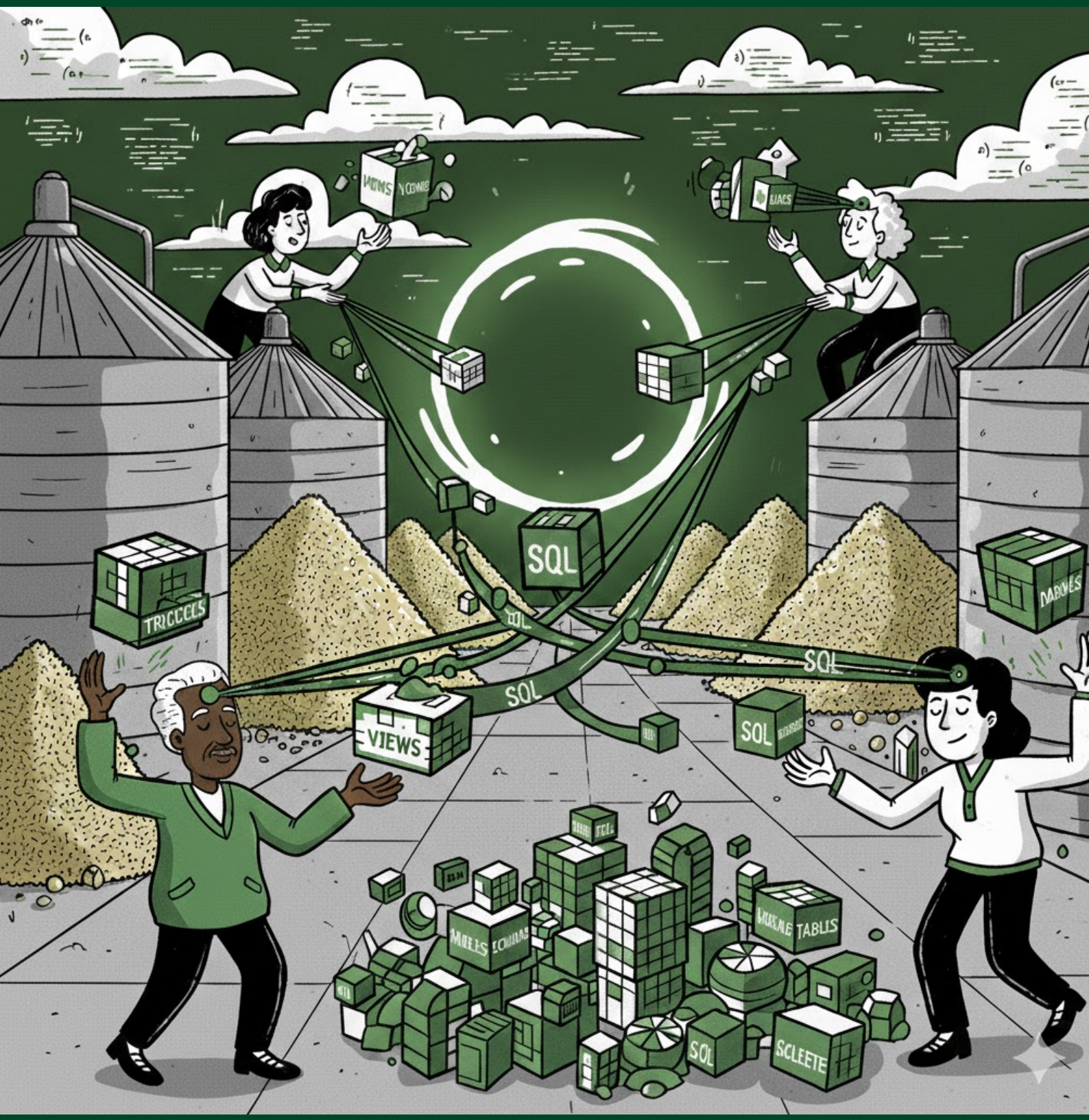
SERGIO SOUZA NOVAK, LEANDRO, RONALDO

Banco de Dados

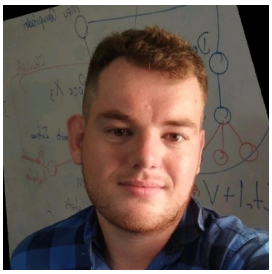
1ª EDIÇÃO



UniRV
Universidade de Rio Verde

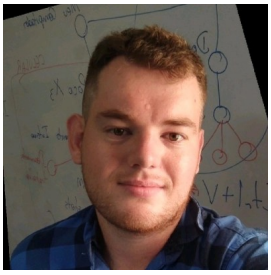
Conheça os Autores

Sergio Souza Novak



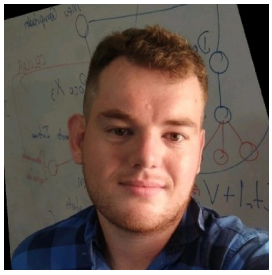
Sergio Souza Novak é Mestre em Ciências Agrárias pelo Instituto Federal Goiano. Atua nas áreas de Banco de Dados, Inteligência Artificial e Desenvolvimento de Software, com experiência acadêmica e industrial e publicações científicas aplicadas à agricultura digital.

Ronaldo



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit.

Leandro Manuel Pereira Sanches



Leandro possui experiência em computação aplicada, análise de sistemas e desenvolvimento tecnológico, participando de projetos acadêmicos e educacionais.

Sumário

1 Modelo Entidade Relacionamento **5**
1.1 Conceito de Atributos . . . 7
1.2 Chave Primária . . . 8
1.3 Conceito de Relacionamentos . . . 9
1.4 Exemplo de diagrama ER . . . 11
1.5 Atividade prática de MER . . . 12
1.5.1 Objetivo . . . 12
1.5.2 Contexto do Sistema . . . 12
1.5.3 Descrição dos Requisitos . . . 12
1.5.4 Regras de Negócio e Relacionamentos . . . 13
1.5.5 Atividades Propostas . . . 14
1.5.6 Critérios de Avaliação . . . 14

2 Introdução ao SQL e ao pgAdmin 4 **15**
2.1 pgAdmin 4 . . . 16
2.2 Iniciando o PostgreSQL no Windows . . . 17
2.3 Método 1: Gerenciador de Serviços (Interface Gráfica) . . . 17
2.4 Método 2: Linha de Comando (Prompt/PowerShell) . . . 18
2.4.1 Iniciar o Serviço . . . 18
2.4.2 Parar o Serviço . . . 18
2.5 Verificação de Variáveis de Ambiente . . . 18
2.6 Teste de Conexão . . . 18
2.7 A Porta Padrão (5432) . . . 18
2.8 Credenciais de Acesso . . . 19
2.9 Comando para Criação do Usuário . . . 19
2.10 String de Conexão (URL) . . . 19
2.11 Atividade Prática: Modelagem de Dados . . . 19
2.11.1 Estrutura Lógica da Tabela `pessoa` . . . 19
2.12 Script SQL de Criação . . . 21
2.13 Operações Básicas (DML) . . . 21
2.13.1 Inserção de Dados . . . 21
2.13.2 Consulta . . . 21
2.14 Comando SELECT: Estrutura e Funcionalidades . . . 22
2.14.1 Praticando Filtros e Ordenação: WHERE e ORDER BY . . . 23
2.14.2 A Cláusula `WHERE` (Onde) . . . 23
2.14.3 A Cláusula `ORDER BY` (Ordenar Por) . . . 24
2.14.4 Restringindo e Agrupando Resultados: LIMIT e HAVING . . . 25
2.15 Estudo Dirigido: Agrupamentos e a Cláusula `HAVING` . . . 26

2.15.1 Passo 1: Criando o Cenário de Teste 26
2.15.2 Passo 2: Populando a Tabela 27
2.15.3 Passo 3: A Consulta de Faturamento 27
2.15.4 Diferença Crucial: WHERE vs HAVING 28
2.16 Análise Avançada: Funções de Agregação e Comportamento do Agrupamento 28
2.16.1 Script de Preparação 28
2.16.2 Discussão: COUNT vs SUM 28
2.16.3 O que acontece se alterarmos para SUM? 29
2.17 Tipos de Dados Básicos no PostgreSQL 29
2.17.1 Principais Categorias 29
2.17.2 Diferenças Importantes para o Desenvolvedor 30
2.17.3 Tipos Numéricos com Precisão 30
2.18 Criação de Tabelas: O Comando `CREATE TABLE` 30
2.18.1 Sintaxe Básica 31
2.18.2 Restrições de Integridade (*Constraints*) 31
2.18.3 Exemplo Comentado 31
2.19 Inserção de Dados: O Comando `INSERT INTO` 32
2.19.1 Sintaxe Básica 32
2.19.2 Regras Cruciais para Inserção 32
2.19.3 Prática: Inserindo Dados na Tabela Pessoa 32

3 Normalização **35**
3.1 Anomalias de Dados 35
3.1.1 Anomalia de Inserção 36
3.1.2 Anomalia de Atualização 37
3.1.3 Anomalia de Exclusão 37
3.2 Dependências Funcionais 38
3.2.1 Dependência Funcional Transitiva 39
3.2.2 Dependência Funcional Total e Dependência Funcional Parcial . . . 40
3.2.3 Dependência Funcional Trivial e Não Trivial 41
3.3 O Processo de Normalização e as Formas Normais mais utilizadas 42
3.3.1 Forma Não Normalizada (FNN ou 0FN) 43
3.3.2 Primeira Forma Normal (1FN) 44
3.3.3 Segunda Forma Normal (2FN) 46
3.3.4 Terceira Forma Normal (3FN) 48
3.4 Formas Normais Avançadas 50
3.4.1 Forma Normal de Boyce-Codd (BCNF) 50
3.4.2 Quarta Forma Normal (4FN) 53
3.4.3 Quinta Forma Normal (5FN ou PJNF) 55
3.5 Resumo Prático de Normalização 57
3.6 Exercícios Propostos 62

Capítulo 1

Modelo Entidade Relacionamento

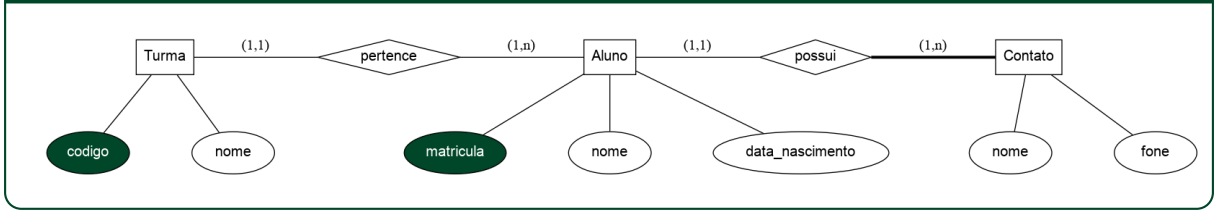
O desenvolvimento de um Banco de Dados para uma aplicação, antes da implementação técnica, exige uma compreensão clara e estruturada dos dados que serão manipulados. Nesse contexto, o **Diagrama Entidade-Relacionamento (DER)** desempenha um papel fundamental no processo de modelagem de dados. O Diagrama Entidade-Relacionamento é um **modelo conceitual**, utilizado para representar, de forma abstrata e organizada, a estrutura das informações de um sistema. Ele está mais próximo da visão que o usuário possui dos dados, pois descreve os elementos do mundo real que serão armazenados e como eles se conectam entre si.

Por meio do DER, é possível identificar e definir:

- Quais dados devem ser armazenados pela aplicação;
- As entidades que representam objetos ou conceitos do domínio do problema;
- Os atributos que caracterizam essas entidades;
- Como esses dados se relacionam entre si.

Dessa forma, o modelo Entidade-Relacionamento serve como base para a construção do modelo lógico e, posteriormente, do banco de dados físico. Ele contribui para reduzir ambiguidades, melhorar a comunicação entre desenvolvedores e usuários e garantir que o sistema atenda corretamente aos requisitos estabelecidos. Ao longo deste livro, serão apresentados os conceitos fundamentais do modelo Entidade-Relacionamento, suas notações, regras de construção e exemplos práticos, permitindo ao leitor compreender e aplicar essa importante ferramenta no projeto de bancos de dados. Abaixo está ilustrado um diagrama desse tipo:

Diagrama ER



Antes de entendermos o diagrama, vejamos a Tabela 1.1. A Tabela 1.1 apresenta os principais elementos que compõem o Diagrama Entidade-Relacionamento, destacando suas representações gráficas e respectivas funções no modelo conceitual. Os retângulos

identificam os conjuntos de entidades, as elipses representam os atributos que descrevem suas características, os losangos indicam os relacionamentos estabelecidos entre entidades e as linhas evidenciam as conexões estruturais entre esses componentes. Em conjunto, esses elementos permitem a representação organizada e formal das informações de um domínio, servindo como base para a modelagem lógica e posterior implementação em banco de dados.

Elemento Gráfico	**Descrição**
Retângulo	Representa um *conjunto de entidades*. Uma entidade corresponde a um objeto ou conceito do mundo real sobre o qual se deseja armazenar informações. Exemplos incluem *Aluno*, *Turma* ou *Produto*. Cada instância da entidade possui existência própria dentro do sistema.
Elipse	Representa um *atributo*. Os atributos descrevem características ou propriedades das entidades ou dos relacionamentos. Exemplos incluem *nome*, *matrícula*, *data_nascimento*. Um atributo pode ser simples, composto, multivalorado ou chave.
Losango	Representa um *relacionamento*. O relacionamento descreve a associação existente entre dois ou mais conjuntos de entidades. Por exemplo, o relacionamento *pertence* pode associar a entidade *Aluno* à entidade *Turma*.
Linha	Representa a ligação entre elementos do diagrama. As linhas conectam atributos às entidades e entidades aos relacionamentos, indicando participação e associação estrutural no modelo conceitual.

Tabela 1.1: Elementos fundamentais do Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento apresentado modela um cenário acadêmico composto pelas entidades *Turma*, *Aluno* e *Contato*. O modelo entidade-relacionamento tem por base a percepção de que o mundo real é formado por conjunto de objetos chamados entidades e pelo conjunto dos relacionamentos entre esses objetos. Retângulos, que representam os conjuntos de entidades; Elipses, que representam os atributos; Losangos, que representam os relacionamentos entre os conjuntos de entidades; Linhas, que unem os atributos aos conjuntos de entidades e o conjunto de entidades aos seus relacionamentos.

O modelo tem como objetivo representar a organização de alunos em turmas e o cadastro de contatos associados a cada aluno, evidenciando as estruturas de dados e as restrições de cardinalidade existentes no domínio analisado. A entidade **Turma** representa as unidades organizacionais às quais os alunos estão vinculados. Ela possui dois atributos: *codigo*, que atua como chave primária e identifica unicamente cada turma no sistema, e *nome*, que descreve a denominação da turma. A presença de uma chave primária garante a integridade da identificação das instâncias dessa entidade. A entidade **Aluno** representa os estudantes vinculados às turmas. Seus atributos são *matricula*,

nome e *data_nascimento*. O atributo *matricula* é definido como chave primária, sendo responsável por identificar de forma única cada aluno. Os demais atributos armazenam informações descritivas e complementares necessárias para caracterizar cada estudante.

A entidade **Contato** representa os dados de contato associados aos alunos. Ela possui os atributos *nome* e *fone*, que registram, respectivamente, o nome do contato e o número de telefone correspondente. Essa entidade permite armazenar múltiplos registros vinculados a um mesmo aluno. O relacionamento **pertence** estabelece a associação entre as entidades *Aluno* e *Turma*. A cardinalidade indicada no diagrama demonstra que cada aluno deve estar obrigatoriamente vinculado a exatamente uma turma, enquanto uma turma pode estar associada a um ou vários alunos. Trata-se, portanto, de um relacionamento do tipo um-para-muitos (1:N), com participação obrigatória do lado da entidade Aluno. O relacionamento **possui** associa a entidade *Aluno* à entidade *Contato*. A cardinalidade apresentada indica que cada aluno possui pelo menos um contato, podendo possuir vários, enquanto cada contato está vinculado a exatamente um aluno. Esse relacionamento também é classificado como um relacionamento do tipo um-para-muitos (1:N), com participação obrigatória da entidade Contato em relação ao aluno ao qual está associado.

De maneira geral, o modelo representa de forma consistente as restrições do domínio acadêmico considerado, garantindo identificação única das entidades principais e estabelecendo vínculos obrigatórios entre alunos e turmas, bem como entre alunos e seus respectivos contatos. A estrutura apresentada é adequada para posterior transformação em modelo relacional, preservando as chaves primárias e as dependências representadas pelas cardinalidades.

1.1 Conceito de Atributos

No modelo Entidade-Relacionamento, os **atributos** são as propriedades que descrevem as características de uma entidade ou de um relacionamento. Eles representam as informações que desejamos armazenar sobre cada ocorrência de uma entidade no sistema.

Cada entidade possui um conjunto próprio de atributos que definem seus dados. Por exemplo, a entidade **Professor** pode possuir atributos como matrícula, nome e telefone. A entidade **Disciplina** pode possuir atributos como código e nome. Já a entidade **Aluno** pode possuir atributos como matrícula, nome e data de nascimento.

Os atributos podem ser classificados em diferentes tipos conceituais:

- **Atributo simples**: não pode ser subdividido em partes menores (exemplo: matrícula).
- **Atributo composto**: pode ser dividido em subpartes (exemplo: um endereço poderia ser dividido em rua, número e cidade).
- **Atributo multivalorado**: pode assumir mais de um valor para a mesma entidade (exemplo: múltiplos telefones).
- **Atributo identificador (chave)**: atributo responsável por identificar unicamente cada ocorrência de uma entidade.

Os atributos são fundamentais para dar significado às entidades, pois sem eles não seria possível armazenar ou diferenciar as instâncias no banco de dados.

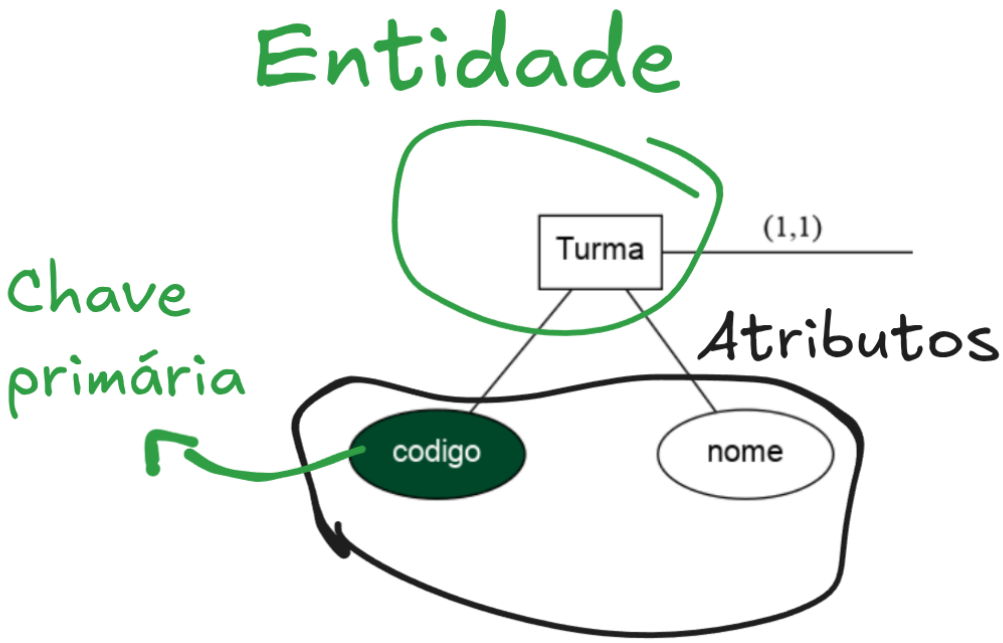


Figura 1.1: Atributos do Diagrama ER.

1.2 Chave Primária

Em um modelo de banco de dados relacional, a **chave primária** é o atributo, ou conjunto de atributos, responsável por identificar de maneira única cada registro de uma tabela. Isso significa que não podem existir duas linhas com o mesmo valor para a chave primária, garantindo que cada tupla seja distinta dentro da relação.

A principal função da chave primária é assegurar a integridade dos dados, evitando duplicidades e permitindo que cada registro seja referenciado de forma precisa. Para cumprir esse papel, ela deve apresentar algumas características fundamentais: seus valores devem ser únicos em toda a tabela, não podem assumir o valor `NULL`, devem ser preferencialmente estáveis ao longo do tempo (ou seja, não devem sofrer alterações frequentes) e devem conter apenas os atributos estritamente necessários para identificar o registro, preservando o princípio da minimalidade.

Como exemplo, considere a entidade *Cliente*, composta pelos atributos id_cliente, nome e telefone. Nesse caso, o atributo `id_cliente` é definido como chave primária, pois identifica exclusivamente cada cliente no sistema, independentemente de possíveis repetições de nome ou telefone.

Em determinadas situações, a chave primária pode ser formada por mais de um atributo, caracterizando uma **chave primária composta**. Isso ocorre quando nenhum atributo isoladamente é capaz de garantir a identificação única do registro. Um exemplo comum é uma tabela que representa um relacionamento muitos-para-muitos entre as entidades *Aluno* e *Disciplina*. Nessa situação, a combinação dos atributos `id_aluno` e `id_disciplina` forma a chave primária, pois apenas o conjunto desses dois valores assegura a unicidade de cada registro.

Assim, a definição adequada da chave primária é essencial no modelo relacional, pois possibilita o estabelecimento correto de relacionamentos entre tabelas, previne inconsis-

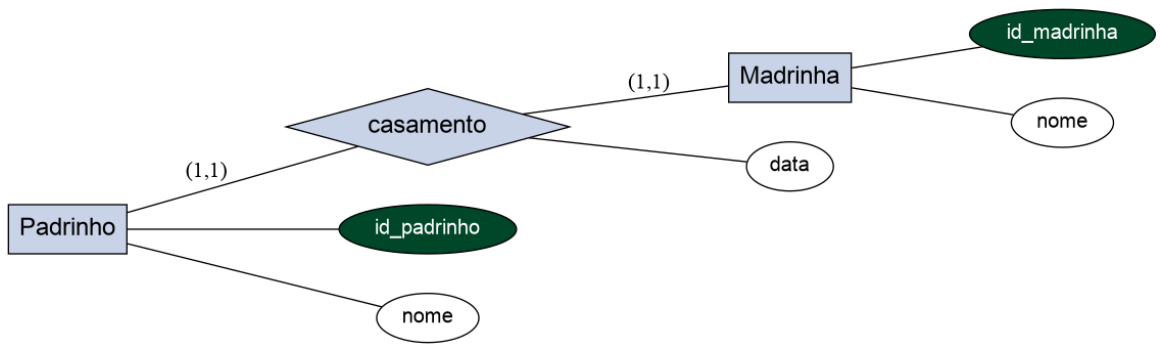
tências e mantém a integridade referencial do banco de dados. Por esse motivo, toda tabela deve possuir uma chave primária claramente definida.

1.3 Conceito de Relacionamentos

Os **relacionamentos** representam as associações existentes entre duas ou mais entidades dentro do modelo. Eles indicam como as entidades se conectam e interagem no domínio do problema. Um relacionamento pode ser classificado de acordo com sua **cardinalidade**, que define o número mínimo e máximo de ocorrências de uma entidade que podem estar associadas a outra. As cardinalidades mais comuns são:

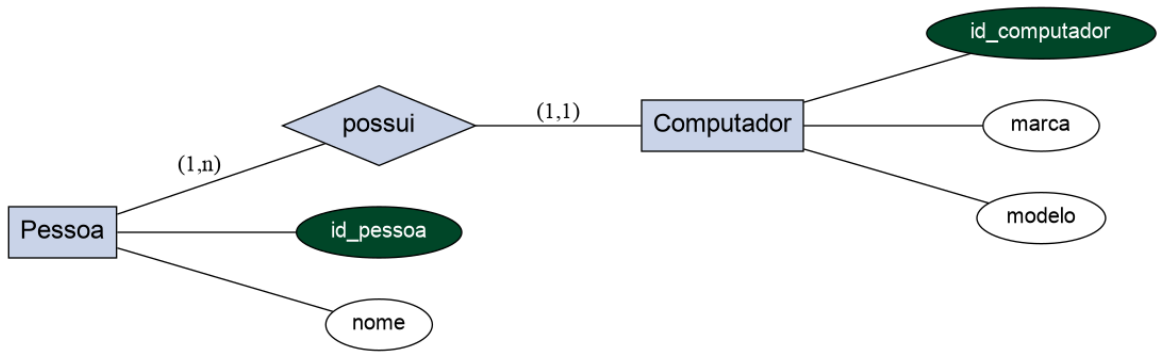
- **Um-para-um (1:1)**: uma ocorrência de uma entidade está associada a apenas uma ocorrência da outra.
- **Um-para-muitos (1:N)**: uma ocorrência de uma entidade pode estar associada a várias ocorrências da outra.
- **Muitos-para-muitos (N:M)**: várias ocorrências de uma entidade podem estar associadas a várias ocorrências da outra.

Diagrama ER



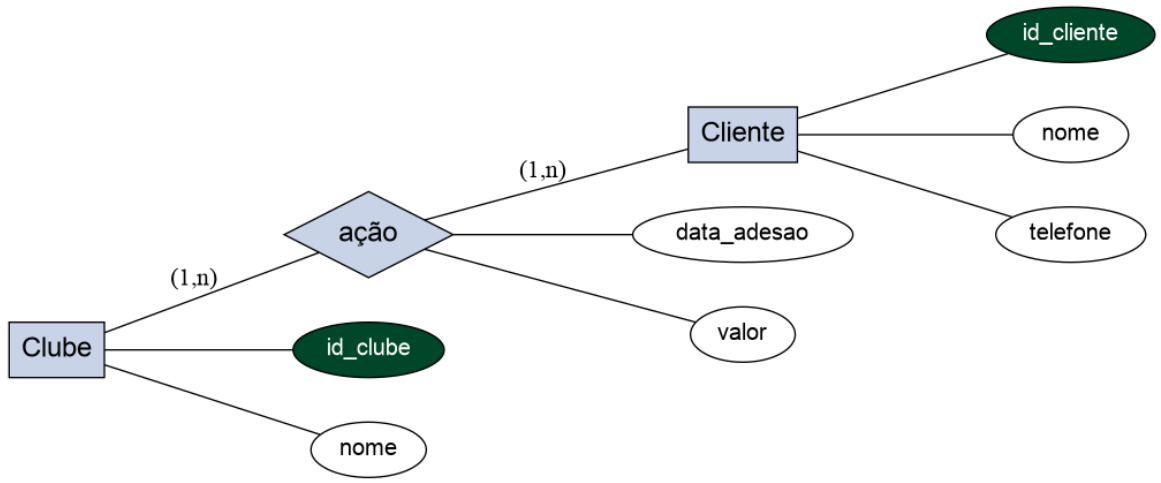
No primeiro diagrama, é apresentado um relacionamento do tipo um-para-um (1:1) entre as entidades Padrinho e Madrinha, mediado pelo relacionamento casamento. A cardinalidade (1,1) em ambos os lados indica que cada padrinho está associado a exatamente uma madrinha, e cada madrinha está associada a exatamente um padrinho dentro do contexto modelado. Isso caracteriza uma associação exclusiva entre as duas entidades. Cada entidade possui um atributo identificador destacado como chave primária id_padrinho e id_madrinha, cuja função é garantir a identificação única de cada ocorrência. Além disso, ambas possuem atributos descritivos, como nome. O relacionamento casamento também possui um atributo próprio, data, que armazena uma informação específica da associação entre padrinho e madrinha, demonstrando que relacionamentos também podem possuir atributos quando é necessário registrar características do vínculo entre as entidades.

Diagrama ER



No segundo diagrama, é representado um relacionamento do tipo um-para-muitos (1:N) entre as entidades Pessoa e Computador, por meio do relacionamento possui. A cardinalidade (1,n) do lado de Pessoa indica que uma pessoa pode possuir vários computadores, enquanto a cardinalidade (1,1) do lado de Computador indica que cada computador pertence a exatamente uma pessoa. Esse tipo de modelagem é comum quando um elemento principal pode estar associado a várias ocorrências de outro elemento dependente. A entidade Pessoa possui como chave primária o atributo id_pessoa, além do atributo descritivo nome. A entidade Computador possui id_computador como chave primária, além dos atributos marca e modelo. Neste caso, o relacionamento não possui atributos próprios, pois a associação "possui" não necessita armazenar informações adicionais além da ligação entre as entidades.

Diagrama ER



No terceiro diagrama, é apresentado um relacionamento do tipo muitos-para-muitos (N:M) entre as entidades Clube e Cliente, por meio do relacionamento ação. A cardinalidade $1, n$ em ambos os lados indica que um clube pode estar associado a vários clientes e que um cliente pode participar de vários clubes simultaneamente. Esse tipo de relacionamento representa uma associação múltipla em ambas as direções, sendo bastante comum em sistemas reais.
A entidade Clube possui id_clube como chave primária e nome como atributo descritivo, enquanto a entidade Cliente possui id_cliente como chave primária, além de nome e telefone. Diferentemente do caso anterior, o relacionamento ação possui atributos próprios, como data_adesao e valor, que armazenam informações específicas da participação do cliente no clube.
Isso demonstra que, em relacionamentos muitos-para-muitos, é frequente a necessidade de registrar dados adicionais referentes à própria associação.

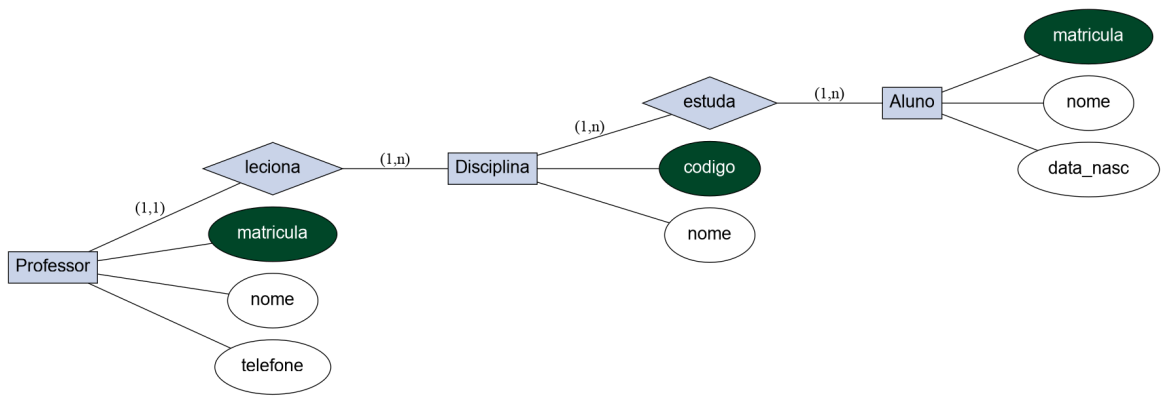
Além disso, os relacionamentos podem possuir atributos próprios, quando é necessário armazenar informações específicas sobre a associação entre as entidades. Portanto, enquanto as entidades representam os objetos principais do domínio e os atributos descrevem suas características, os relacionamentos estabelecem as ligações estruturais que organizam o modelo de dados.

1.4 Exemplo de diagrama ER

O diagrama Entidade-Relacionamento apresentado é composto por três entidades principais: **Professor**, **Disciplina** e **Aluno**. Essas entidades representam os objetos fundamentais do domínio acadêmico modelado. A entidade **Professor** representa os docentes responsáveis pela oferta das disciplinas. Já a entidade **Disciplina** representa as matérias ofertadas na instituição. Por sua vez, a entidade **Aluno** representa os estudantes matriculados na instituição. Entre essas entidades existem dois relacionamentos principais. O primeiro relacionamento, denominado **leciona**, conecta as entidades **Professor** e **Disciplina**. A cardinalidade indica que cada professor participa do relacionamento com cardinalidade (1,1), enquanto cada disciplina participa com cardinalidade (1,n). Isso significa que um professor está associado a exatamente uma ocorrência no relacionamento, enquanto uma disciplina pode estar associada a uma ou várias ocorrências conforme definido no modelo. O segundo relacionamento, denominado **estuda**, conecta as entidades **Disciplina** e **Aluno**. A cardinalidade (1,n) em ambos os lados indica que uma disciplina pode estar relacionada a vários alunos e que um aluno pode estar relacionado a várias disciplinas, caracterizando um relacionamento do tipo muitos-para-muitos.

Assim, o modelo descreve a estrutura básica de um sistema acadêmico no qual professores lecionam disciplinas e alunos estudam disciplinas, estabelecendo as ligações essenciais entre os principais elementos do domínio.

Diagrama ER



1.5 Atividade prática de MER

1.5.1 Objetivo

O objetivo desta atividade é desenvolver a capacidade de análise de requisitos e modelagem conceitual de um banco de dados por meio da construção de um **Diagrama Entidade-Relacionamento (DER)** para um sistema realista de controle de ordenha em uma fazenda leiteira.

1.5.2 Contexto do Sistema

Figura 1.2: Uma fazenda da cidade de Rio Verde. Fonte: Instagram @fazendanycolle

Uma fazenda leiteira deseja informatizar o controle de suas atividades de ordenha. Atualmente, os registros são feitos manualmente, o que dificulta o acompanhamento da produção, da saúde dos animais e do desempenho dos funcionários. O sistema deverá armazenar informações sobre as vacas, as ordenhas realizadas, os funcionários responsáveis, os equipamentos utilizados e a produção de leite obtida em cada ordenha.

1.5.3 Descrição dos Requisitos

Abaixo são detalhados os requisitos levantados para a modelagem do sistema:

Vacas

Cada vaca da fazenda deve ser identificada por um código único. Além disso, devem ser armazenadas informações como:

- Nome ou identificação visual;
- Data de nascimento;
- Raça, Peso e Status de saúde.

Funcionários e Supervisão

A fazenda possui funcionários com diferentes atribuições:

- **Funcionários de Campo:** Registrados por código, nome completo, função (ordenhador, etc.) e telefone.
- **Supervisor:** Responsável por acompanhar o trabalho. Registra-se código, nome, telefone e turno de supervisão.
- **Veterinário:** Responsável pela saúde clínica. Registra-se código, nome, registro profissional (CRMV) e telefone.

Processo de Ordenha

A ordenha ocorre diariamente e pode acontecer mais de uma vez por dia para a mesma vaca. Cada registro de ordenha deve conter: **Data, Horário e Quantidade de leite (litros)**.

1.5.4 Regras de Negócio e Relacionamentos

Para facilitar o entendimento das cardinalidades, o produtor descreveu a rotina da seguinte forma:

Rotina de Ordenha

Cada vaca da fazenda é ordenhada ao longo do tempo. Em alguns períodos, a mesma vaca pode ser ordenhada mais de uma vez no mesmo dia. Sempre que ocorre uma ordenha, ela diz respeito a uma única vaca.

Trabalho dos Funcionários

Os funcionários são responsáveis por realizar as ordenhas. Um mesmo funcionário pode ordenhar várias vacas ao longo do dia. No entanto, toda ordenha precisa ter alguém responsável por realizá-la.

Uso dos Equipamentos

Para realizar a ordenha, podem ser usados um ou vários equipamentos (manuais, automáticos ou robóticos). Os equipamentos podem ser utilizados em diferentes ordenhas ou ficar parados para manutenção.

Cuidado com a Saúde

O veterinário pode atender várias vacas ao longo do tempo e, dependendo do caso, a mesma vaca pode precisar de mais de um atendimento em momentos diferentes.

Visão do Proprietário

O proprietário não participa da ordenha diretamente, mas acompanha os relatórios de produção, desempenho da equipe e utilização de equipamentos para tomada de decisão.

1.5.5 Atividades Propostas

Com base na descrição acima, realize a modelagem conceitual seguindo os passos:

1. Identifique todas as **entidades** do sistema.
2. Para cada entidade, defina seus **atributos** e a **chave primária**.
3. Modele os **relacionamentos** (ex: Vaca *sofre* Ordenha).
4. Defina as **cardinalidades** (1:1, 1:N, N:M) e a **participação** (total ou parcial).
5. Identifique se há necessidade de atributos nos relacionamentos.

Instruções de Entrega: O Diagrama Entidade-Relacionamento deve ser feito de forma legível (à mão ou ferramenta digital como brModelo/Lucidchart). O arquivo final deve ser convertido para **PDF** e conter apenas o diagrama finalizado.

1.5.6 Critérios de Avaliação

A avaliação considerará a correta identificação das entidades, adequação das chaves primárias, precisão das cardinalidades frente às regras de negócio e a organização visual do diagrama.

Capítulo 2

Introdução ao SQL e ao pgAdmin 4

SQL é a sigla para *Structured Query Language* (Linguagem de Consulta Estruturada). Trata-se de uma linguagem padrão utilizada para manipular e gerenciar bancos de dados relacionais. Um **banco de dados relacional** organiza as informações em **tabelas**, compostas por **linhas** (registros) e **colunas** (atributos). Esse modelo foi formalizado por Edgar F. Codd na década de 1970 e tornou-se o paradigma dominante em sistemas de gerenciamento de dados. A linguagem SQL permite realizar diversas operações fundamentais no gerenciamento de bancos de dados relacionais, tais como a criação de bancos de dados e tabelas, a inserção de novos registros, a consulta de informações específicas por meio de critérios definidos, a atualização de dados já existentes, a remoção de registros e o controle de permissões de acesso dos usuários ao sistema.É importante destacar que o SQL é utilizado por diversos Sistemas Gerenciadores de Banco de Dados (SGBDs), como o *PostgreSQL*, o MySQL, o Oracle Database e o SQL Server.

De forma didática, podemos dividir os comandos SQL em categorias:

- **DDL (Data Definition Language)**: comandos de definição de estrutura, como `CREATE`, `ALTER` e `DROP`.
- **DML (Data Manipulation Language)**: comandos de manipulação de dados, como `INSERT`, `UPDATE`, `DELETE`.
- **DQL (Data Query Language)**: comandos de consulta, como `SELECT`.
- **DCL (Data Control Language)**: comandos de controle de acesso, como `GRANT` e `REVOKE`.

Um exemplo simples de consulta em SQL é:

SQL

```sql
SELECT nome, idade
FROM alunos
WHERE idade > 18;
```

Nesse exemplo, o comando solicita a exibição dos campos `nome` e `idade` da tabela `alunos`, considerando apenas os registros em que a idade seja maior que 18 anos. Esse exemplo poderia retornar a tabela abaixo, dependendo dos dados armazenados:

Nome	Idade	CPF
Ana Silva	20	123.456.789-00
Carlos Lima	22	987.654.321-00
Mariana Souza	19	456.789.123-00

2.1 pgAdmin 4

O pgAdmin 4 é uma ferramenta gráfica utilizada para administrar bancos de dados do sistema PostgreSQL. Ele funciona como uma interface visual que facilita a interação com o banco de dados, especialmente para usuários iniciantes, pois permite executar tarefas administrativas sem a necessidade de memorizar todos os comandos da linguagem SQL. A Figura 2.1 apresenta a interface do pgAdmin 4, destacando os principais painéis e recursos disponíveis para gerenciamento de bancos de dados PostgreSQL.

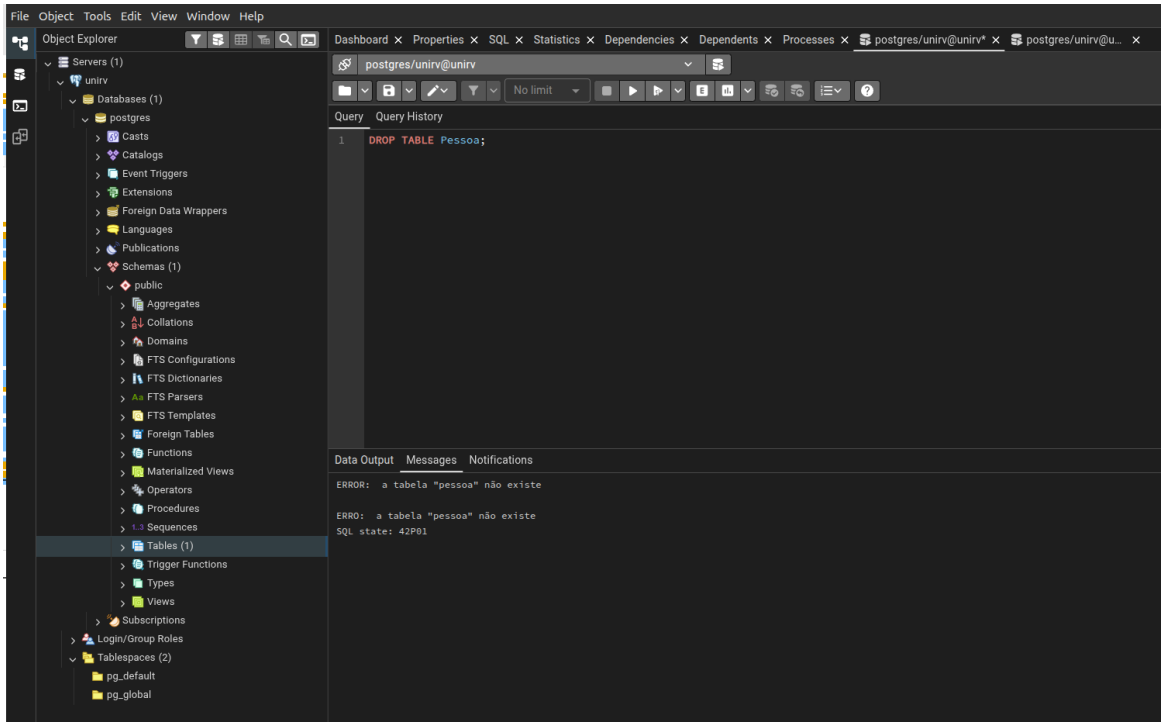


Figura 2.1: Interface do pgAdmin 4.

Enquanto o SQL é a linguagem utilizada para escrever comandos e realizar consultas diretamente no banco de dados, o pgAdmin 4 é o ambiente que possibilita criar e configurar servidores PostgreSQL, criar bancos de dados, tabelas e relacionamentos por meio de recursos gráficos, executar comandos SQL em um editor integrado, visualizar e editar dados em formato tabular, gerenciar usuários e permissões e monitorar as atividades do servidor.

O pgAdmin 4 pode ser executado tanto como aplicação desktop quanto por meio de navegador web. Sua interface é organizada em painéis, normalmente estruturados em um navegador de objetos (que apresenta a árvore de bancos de dados e seus componentes), um editor de consultas (Query Tool), no qual os comandos SQL são digitados e executados, e uma área destinada à exibição de resultados e mensagens do sistema.

Figura 2.2: Comparação entre pgAdmin 4 e PostgreSQL.

Há de se ressaltar que o pgAdmin 4 é uma ferramenta de administração e desenvolvimento para bancos de dados PostgreSQL, enquanto o PostgreSQL é o sistema gerenciador de banco de dados relacional em si. O pgAdmin 4 serve como uma interface gráfica para facilitar a interação com o PostgreSQL, permitindo que os usuários criem, configurem e gerenciem seus bancos de dados de maneira mais intuitiva, sem a necessidade de utilizar exclusivamente a linha de comando ou scripts SQL.

2.2 Iniciando o PostgreSQL no Windows

No ambiente Windows, o PostgreSQL é gerenciado como um serviço do sistema. Muitas vezes, após a instalação ou um reinício, o serviço pode estar interrompido, impedindo a conexão via pgAdmin ou terminal.

2.3 Método 1: Gerenciador de Serviços (Interface Gráfica)

Este é o método mais simples para usuários que preferem não utilizar linhas de comando.

1. Pressione as teclas **Win** + **R**, digite `services.msc` e aperte Enter.
2. Na lista de serviços, procure por **postgresql-x64-[versão]** (ex: postgresql-x64-16).
3. Clique com o botão direito sobre o nome do serviço.
4. Selecione **Iniciar** (Start) ou **Reiniciar** (Restart).

2.4 Método 2: Linha de Comando (Prompt/PowerShell)

Para ativar o serviço via terminal, você deve executar o Prompt de Comando como **Administrador**.

2.4.1 Iniciar o Serviço

SQL

```
net start postgresql-x64-16
```

2.4.2 Parar o Serviço

SQL

```
net stop postgresql-x64-16
```

**Nota: Substitua "16"pela versão que você instalou.*

2.5 Verificação de Variáveis de Ambiente

Para utilizar o comando `psql` em qualquer lugar do terminal, o caminho da pasta `bin` do PostgreSQL deve estar no **PATH** do sistema.

- Caminho padrão: `C:\Program Files\PostgreSQL\16\bin`

2.6 Teste de Conexão

Após ativar o serviço, teste a conexão no terminal:

SQL

```
psql -U postgres
```

O sistema solicitará a senha definida durante a instalação.

2.7 A Porta Padrão (5432)

O PostgreSQL utiliza, por padrão, a porta **5432** para comunicações TCP/IP. Esta porta é o "endereço do guichê"onde o serviço escuta as requisições de clientes como o pgAdmin, DBeaver ou aplicações Java/Python.

- **Localização do arquivo:** A porta pode ser alterada no arquivo `postgresql.conf`, na linha `port = 5432`.
- **Firewall:** Certifique-se de que a porta 5432 está liberada nas regras de entrada do Windows se for acessar o banco de outra máquina.

2.8 Credenciais de Acesso

Para este ambiente, utilizaremos as seguintes credenciais:

Parâmetro	Valor
Usuário	`unirv`
Senha	`unirv`
Porta	5432
Host	localhost

Tabela 2.1: Resumo das credenciais de conexão.

2.9 Comando para Criação do Usuário

Como o PostgreSQL inicia apenas com o usuário `postgres`, é necessário executar o comando abaixo para habilitar o novo usuário:

SQL

```sql
-- Criar usuario com senha e privilégios de criação de banco
CREATE USER unirv WITH PASSWORD 'unirv' CREATEDB;

-- Caso queira que ele seja um superusuario (uso acadêmico)
ALTER USER unirv WITH SUPERUSER;
```

2.10 String de Conexão (URL)

Em muitas linguagens de programação, a conexão é feita através de uma única linha:

`postgresql://unirv:unirv@localhost:5432/nome_do_banco`

2.11 Atividade Prática: Modelagem de Dados

Exercício Dirigido com o Professor

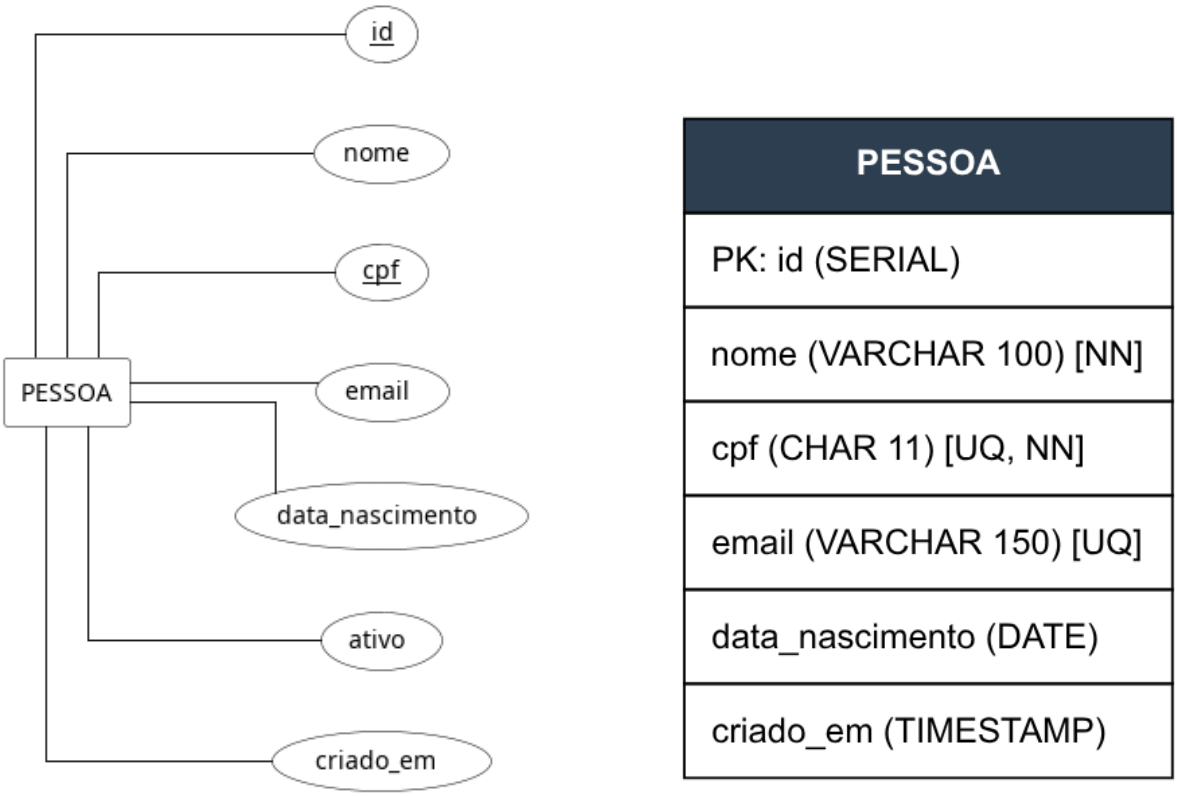
Nesta etapa, realizaremos a modelagem lógica da nossa primeira entidade. Sob a orientação do professor, analise os atributos necessários para representar uma **Pessoa** no sistema.
O objetivo é compreender como as regras de negócio do mundo real se transformam em restrições (*constraints*) no PostgreSQL.

2.11.1 Estrutura Lógica da Tabela `pessoa`

Abaixo, detalhamos a estrutura que será implementada. Preste atenção aos tipos de dados e às propriedades de cada campo:

- **id**: Chave Primária (*Primary Key*). Será do tipo `SERIAL` para garantir que o identificador seja autoincrementado e exclusivo.
- **nome**: Nome completo. Campo **obrigatório** (*NOT NULL*), definido como `VARCHAR(100)`.
- **cpf**: Cadastro de Pessoa Física. Deve ser **único** (*UNIQUE*) e **obrigatório**, garantindo que não haja duplicidade de registros.
- **email**: Endereço eletrônico. Também deve ser **único**, permitindo o uso como login ou canal de comunicação exclusivo.
- **data_nascimento**: Campo do tipo `DATE`, essencial para o cálculo de idade e validações de maioridade.
- **criado_em**: Registro automático de auditoria. Armazena a data e hora exata em que o registro foi inserido no banco.

Nota para o aluno: *Durante a explicação do professor, anote quais desses campos poderiam ser opcionais em um cenário real e quais devem ser sempre protegidos por restrições de integridade.*



(a) Diagrama ER (b) Diagrama Lógico

Figura 2.3: Modelagem da Entidade Pessoa: Visão ER e Lógica.

2.12 Script SQL de Criação

O código abaixo executa a criação da tabela no schema `public`.

SQL

```sql
CREATE TABLE pessoa (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    email VARCHAR(150) UNIQUE,
    data_nascimento DATE,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Adicionar um comentário na tabela para documentação

COMMENT ON TABLE pessoa IS 'Tabela que armazena dados cadastrais de
↪  pessoas físicas';
```

2.13 Operações Básicas (DML)

2.13.1 Inserção de Dados

Para inserir um novo registro, utilize o comando abaixo:

SQL

```sql
INSERT INTO pessoa (nome, cpf, email, data_nascimento)
VALUES ('João Silva', '12345678901', 'joao@email.com', '1990-05-15');
```

2.13.2 Consulta

Para listar todas as pessoas cadastradas:

SQL

```sql
SELECT id, nome, email FROM pessoa WHERE ativo = TRUE;
```

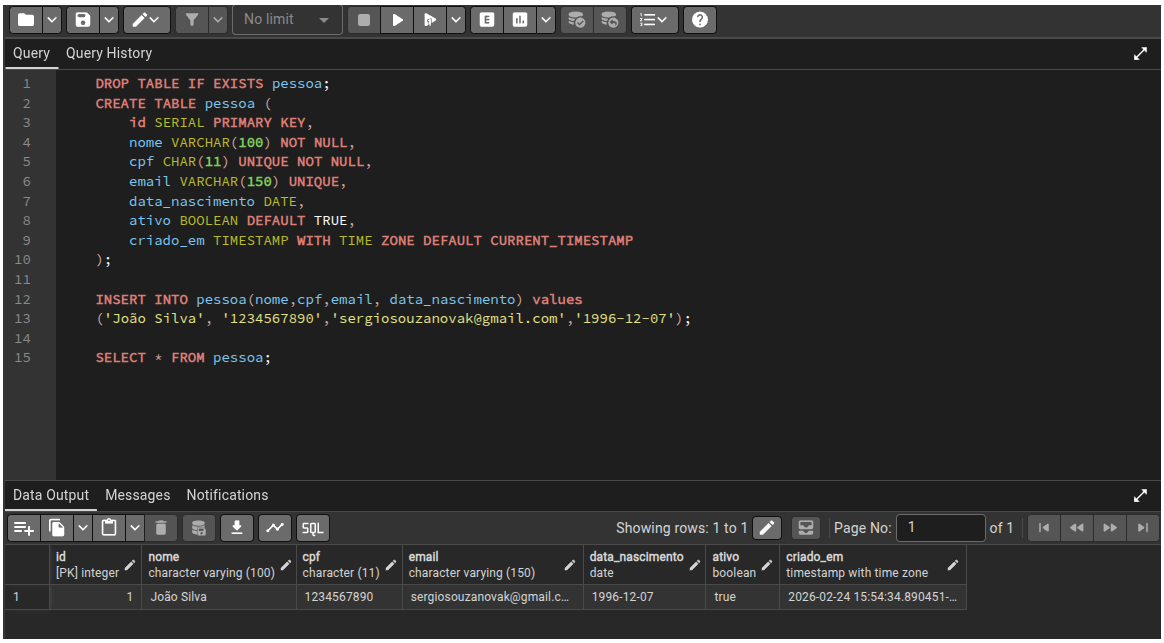


Figura 2.4: Execução do comando SELECT na tabela pessoa.

2.14 Comando SELECT: Estrutura e Funcionalidades

SQL

```sql
SELECT nome, salario
FROM funcionarios
WHERE salario > 30020
ORDER BY salario DESC;
```

O comando `SELECT` é utilizado para realizar consultas em um banco de dados relacional. Sua principal finalidade é recuperar dados armazenados em uma ou mais tabelas, permitindo ao usuário visualizar informações específicas conforme determinados critérios.

A estrutura básica do comando é:

SQL

```sql
SELECT coluna1, coluna2, ...
FROM nome_da_tabela;
```

Nesse formato, especificam-se as colunas que se deseja visualizar após a palavra-chave `SELECT`, enquanto a cláusula `FROM` indica a tabela de onde os dados serão obtidos. Caso seja necessário retornar todas as colunas da tabela, pode-se utilizar o caractere curinga `*`, como no exemplo:

SQL

```sql
SELECT * FROM alunos;
```

Além disso, o comando `SELECT` pode ser combinado com outras cláusulas para refinar

a consulta. Entre as mais utilizadas estão:

- `WHERE`: filtra registros de acordo com uma condição lógica;
- `ORDER BY`: organiza os resultados em ordem crescente ou decrescente;
- `GROUP BY`: agrupa registros para aplicação de funções agregadas;
- `HAVING`: define condições sobre grupos;
- `LIMIT`: restringe a quantidade de registros retornados (em alguns SGBDs).

Por exemplo:

SQL

```sql
SELECT nome, idade
FROM alunos
WHERE idade > 18
ORDER BY idade DESC;
```

Nesse caso, a consulta retorna apenas os alunos com idade superior a 18 anos, organizando o resultado em ordem decrescente de idade.Portanto, o comando `SELECT` é a base das operações de consulta em SQL, sendo essencial para a extração e análise de dados em sistemas de gerenciamento de bancos de dados relacionais.

2.14.1 Praticando Filtros e Ordenação: WHERE e ORDER BY

Exercício Dirigido com o Professor: Refinando a Busca

Em sistemas reais, um banco de dados pode conter milhões de registros. Retornar todos os dados de uma vez não é apenas ineficiente, mas muitas vezes inútil para o usuário final. Sob a orientação do professor, vamos testar na nossa tabela `Pessoa` como extrair apenas a informação exata que precisamos.

2.14.2 A Cláusula `WHERE` (Onde)

A cláusula `WHERE` atua como um "peneira". Ela avalia cada linha da tabela e retorna apenas aquelas que satisfazem uma condição lógica. Podemos usar operadores de comparação como `=`, `>`, `<`, `>=`, `<=` e `<>` (diferente).

Por exemplo, se quisermos encontrar apenas os usuários que estão com o cadastro ativo em nosso sistema:

SQL

```sql
-- O banco de dados retornará apenas as linhas onde 'ativo' for verdadeiro
SELECT nome, email
FROM pessoa
WHERE ativo = TRUE;
```

Outro exemplo comum é o uso com datas. Vamos buscar todas as pessoas que nasceram após o ano 2000:

SQL

```sql
SELECT nome, data_nascimento
FROM pessoa
WHERE data_nascimento > '2000-12-31';
```

2.14.3 A Cláusula `ORDER BY` (Ordenar Por)

Enquanto o `WHERE` decide *quais* linhas serão exibidas, o `ORDER BY` decide *em qual ordem* elas aparecerão na tela. A ordenação pode ser feita em duas direções:

- `ASC` (Ascendente): Ordem crescente (A-Z, 0-9, datas mais antigas para as mais recentes). Se você não especificar a direção, o SQL usará `ASC` por padrão.
- `DESC` (Descendente): Ordem decrescente (Z-A, 9-0, datas mais recentes para as mais antigas).

Se o professor pedir para organizar a lista de pessoas em ordem alfabética inversa (de Z a A), o comando será:

SQL

```sql
SELECT nome, cpf
FROM pessoa
ORDER BY nome DESC;
```

Juntando as Peças

O verdadeiro poder do SQL aparece quando combinamos essas cláusulas. É importante notar que **a ordem dos comandos importa**: o `WHERE` sempre deve vir antes do `ORDER BY`.

Vamos resolver um cenário prático: *"Liste o nome e a data de nascimento de todas as pessoas com cadastro ativo, ordenando das mais velhas para as mais novas"*.

SQL

```sql
SELECT nome, data_nascimento
FROM pessoa
WHERE ativo = TRUE
ORDER BY data_nascimento ASC;
```

2.14.4 Restringindo e Agrupando Resultados: LIMIT e HAVING

Exercício Dirigido com o Professor: Top Resultados e Agrupamentos

À medida que nosso banco de dados cresce, algumas perguntas mudam. Em vez de perguntar "Quais são todos os usuários?", passamos a perguntar "Quais são os 5 usuários mais recentes?"ou "Quantos usuários ativos temos em comparação aos inativos?". Acompanhe com o professor como responder a essas perguntas de negócios.

A Cláusula `LIMIT` (Limite de Linhas)

A cláusula `LIMIT` é extremamente útil para **paginação** (como quando você avança para a "página 2"de uma pesquisa no Google) ou para buscar os resultados do "Top N"(os maiores, menores, mais recentes, etc.). Ela simplesmente corta o resultado final para exibir apenas a quantidade de linhas especificada.

Exemplo Prático: Se o professor pedir para listar apenas as **3 pessoas cadastradas mais recentemente**, precisamos ordenar pela data de criação de forma decrescente e, em seguida, aplicar o limite:

SQL

```sql
-- Traz os 3 registros mais novos
SELECT nome, criado_em
FROM pessoa
ORDER BY criado_em DESC
LIMIT 3;
```

A Cláusula `HAVING` (Filtro de Grupos)

Para entender o `HAVING`, primeiro precisamos lembrar do `GROUP BY` (Agrupar Por), que une registros que possuem valores idênticos em uma ou mais colunas, permitindo o uso de funções matemáticas como `COUNT()` (contar), `SUM()` (somar) ou `AVG()` (média).

A grande confusão dos alunos iniciantes é: *Qual a diferença entre WHERE e HAVING?*

- O `WHERE` filtra **linhas individuais** ANTES de serem agrupadas.
- O `HAVING` filtra **os grupos** DEPOIS que eles foram formados.

Exemplo Prático: Vamos agrupar as pessoas pelo status (`ativo` = `TRUE` ou `FALSE`) e contar quantas pessoas existem em cada grupo. Porém, o professor solicitou que a consulta mostre **apenas os grupos que possuam mais de 10 pessoas**.

Como a restrição "mais de 10"é baseada no resultado de uma contagem (um grupo), **não podemos usar o WHERE**. Usamos o `HAVING`:

SQL

```sql
SELECT ativo, COUNT(id) AS total_pessoas
FROM pessoa
GROUP BY ativo
HAVING COUNT(id) > 10;
```

Atenção à Ordem de Execução!

O banco de dados SQL segue uma ordem estrita para ler o seu comando. Se você for usar todas as cláusulas que aprendemos até agora em uma única consulta, a ordem **obrigatória** no código deve ser:

1. `SELECT` (O que mostrar)
2. `FROM` (De qual tabela)
3. `WHERE` (Filtro de linhas)
4. `GROUP BY` (Como agrupar)
5. `HAVING` (Filtro de grupos)
6. `ORDER BY` (Ordem de exibição final)
7. `LIMIT` (Quantidade final de linhas)

2.15 Estudo Dirigido: Agrupamentos e a Cláusula `HAVING`

Objetivo da Atividade

Neste exercício, vamos simular um sistema de vendas para entender como o SQL pode realizar cálculos complexos e filtrar resultados baseados em agregados (somas), algo que o `WHERE` não consegue fazer sozinho.

2.15.1 Passo 1: Criando o Cenário de Teste

O primeiro passo é criar uma tabela temporária. Tabelas do tipo `TEMP` existem apenas durante a sua sessão atual no banco de dados.

SQL

```sql
-- 1. Criar tabela de vendas rápida
CREATE TEMP TABLE vendas (
    id SERIAL,
    produto VARCHAR(50),
    categoria VARCHAR(50),
    valor DECIMAL(10,2)
);
```

2.15.2 Passo 2: Populando a Tabela

Vamos inserir dados que nos permitam comparar categorias diferentes. Observe que a categoria 'Eletrônicos' terá um valor total maior que 'Papelaria'.

SQL

```sql
-- 2. Inserir dados para teste
INSERT INTO vendas (produto, categoria, valor) VALUES
('Teclado', 'Eletrônicos', 80.00),
('Mouse',   'Eletrônicos', 50.00),  -- Soma Eletrônicos: 130.00
('Caderno', 'Papelaria',   15.00),
('Caneta',  'Papelaria',    5.00);  -- Soma Papelaria: 20.00
```

2.15.3 Passo 3: A Consulta de Faturamento

Agora, execute a query abaixo. O objetivo é listar o faturamento total por categoria, mas **excluir** do relatório qualquer categoria que tenha vendido menos de 100.00 no total.

SQL

```sql
-- 3. Testar a query com HAVING
SELECT categoria, SUM(valor) AS faturamento_total
FROM vendas
GROUP BY categoria
HAVING SUM(valor) > 100.00;
```

Questionário de Reflexão

Após executar os comandos acima, discuta com o professor:

1. Por que a categoria 'Papelaria' não apareceu no resultado final?
2. Se mudarmos o valor do `HAVING` para `10.00`, o que acontece?
3. Tente substituir `HAVING SUM(valor) > 100.00` por `WHERE SUM(valor) > 100.00`. Qual erro o PostgreSQL retorna? Por que isso acontece?

2.15.4 Diferença Crucial: WHERE vs HAVING

Para nunca mais esquecer:

- **WHERE**: Filtra os dados **antes** da soma (ex: excluir o produto 'Caneta' antes de somar).
- **HAVING**: Filtra os dados **depois** da soma (ex: olhar o total da categoria e decidir se mostra ou não).

2.16 Análise Avançada: Funções de Agregação e Comportamento do Agrupamento

Nesta etapa do estudo dirigido, vamos analisar como o banco de dados se comporta quando alteramos a lógica da nossa "pergunta". O código abaixo prepara uma base com diferentes estados de atividade para testarmos a sensibilidade das funções `COUNT` e `SUM`.

2.16.1 Script de Preparação

Observe que estamos lidando com restrições de integridade (*Unique*) e valores padrão (*Default*).

SQL

```sql
DROP TABLE IF EXISTS PESSOA;

CREATE TABLE PESSOA (
    ID SERIAL PRIMARY KEY,
    NOME VARCHAR(100) NOT NULL,
    CPF CHAR(11) UNIQUE NOT NULL,
    EMAIL VARCHAR(150) UNIQUE,
    DATA_NASCIMENTO DATE,
    ATIVO BOOLEAN DEFAULT TRUE,
    CRIADO_EM TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Inserindo dois registros inativos
INSERT INTO PESSOA (NOME, CPF, EMAIL, DATA_NASCIMENTO, ATIVO)
VALUES ('João Silva', '12345678900', 'joao@email.com', '1996-12-07',
↪  false),
       ('Camila Santana', '12345678901', 'camila@email.com', '1996-12-07',
       ↪  false);

-- Inserindo um registro ativo (padrão)
INSERT INTO PESSOA (NOME, CPF, EMAIL, DATA_NASCIMENTO)
VALUES ('João Silv1a', '12345678902', 'joao2@email.com', '1996-12-07');
```

2.16.2 Discussão: COUNT vs SUM

A consulta original utiliza a função `COUNT(ID)`. Mas o que aconteceria se tentássemos usar `SUM`?

SQL

```sql
-- Consulta A: Contagem por grupo
SELECT ATIVO, COUNT(ID) AS TOTAL_PESSOAS
FROM PESSOA
GROUP BY ATIVO;
```

Conceito Fundamental

- **COUNT(ID)**: Conta quantas **linhas** existem no grupo. Funciona com qualquer tipo de dado (números, texto, datas, booleanos). No nosso exemplo, resultará em 2 para *false* e 1 para *true*.
- **SUM(coluna)**: Tenta realizar uma **adição aritmética** dos valores. Só funciona com tipos de dados numéricos (inteiros, decimais).

2.16.3 O que acontece se alterarmos para SUM?

Se tentarmos rodar `SELECT ATIVO, SUM(ID) FROM PESSOA GROUP BY ATIVO;`, o banco somará os valores numéricos das Chaves Primárias.

Função	Lógica	Resultado (para ATIVO = false)
`COUNT(ID)`	1 + 1 (duas ocorrências)	**2**
`SUM(ID)`	ID 1 + ID 2 (soma dos valores)	**3**

Tabela 2.2: Diferença de resultado entre contar e somar.

Reflexão para o Aluno

Pergunta: Por que não poderíamos usar `SUM(NOME)`?

2.17 Tipos de Dados Básicos no PostgreSQL

A escolha do tipo de dado correto é uma das etapas mais importantes da modelagem. No PostgreSQL, cada coluna deve ter um tipo definido para que o SGBD saiba como armazenar e processar aquela informação.

2.17.1 Principais Categorias

Abaixo, apresentamos os tipos mais utilizados em aplicações comerciais e acadêmicas:

Categoria	Tipo SQL	Descrição / Exemplo
Numéricos	`INTEGER`	Números inteiros (ex: idade, quantidade).
	`DECIMAL` ou `NUMERIC`	Números reais com precisão (ex: salário, preço).
	`SERIAL`	Inteiro autoincrementado (usado para IDs).
Texto	`VARCHAR(n)`	Texto de tamanho variável com limite n.
	`CHAR(n)`	Texto de tamanho fixo (ex: CPF, UF).
	`TEXT`	Texto de tamanho ilimitado (ex: descrições).
Data/Hora	`DATE`	Apenas data (AAAA-MM-DD).
	`TIMESTAMP`	Data e hora combinadas.
Lógicos	`BOOLEAN`	Valores verdadeiro (`TRUE`) ou falso (`FALSE`).

Tabela 2.3: Tipos de dados fundamentais no PostgreSQL.

2.17.2 Diferenças Importantes para o Desenvolvedor

Dica do Professor: CHAR vs VARCHAR

Uma dúvida comum é quando usar `CHAR` ou `VARCHAR`.

- Use `CHAR(n)` para dados que **sempre** têm o mesmo tamanho, como um campo *Estado* (UF) que sempre terá 2 letras ou um *CPF* com 11 dígitos.
- Use `VARCHAR(n)` para nomes, e-mails ou endereços, onde o tamanho muda de uma pessoa para outra. Isso economiza espaço de armazenamento.

2.17.3 Tipos Numéricos com Precisão

Para lidar com dinheiro (moeda), nunca utilize tipos de ponto flutuante (*float*), pois eles podem apresentar arredondamentos imprecisos. Utilize sempre:

SQL

```sql
-- Exemplo: Salario com 10 digitos no total, sendo 2 decimais
salario DECIMAL(10,2)
```

2.18 Criação de Tabelas: O Comando `CREATE TABLE`

Após o planejamento no Diagrama Entidade-Relacionamento (ER), o próximo passo é a implementação física no SGBD. Para isso, utilizamos a linguagem de definição de dados (*Data Definition Language* - DDL) através do comando `CREATE TABLE`.

Este comando define não apenas o nome da tabela, mas também quais colunas ela terá, seus tipos de dados e suas regras de integridade.

2.18.1 Sintaxe Básica

A estrutura fundamental do comando segue o padrão abaixo:

SQL

```sql
CREATE TABLE nome_da_tabela (
    coluna1 tipo_de_dado restricoes,
    coluna2 tipo_de_dado restricoes,
    ...
    colunaN tipo_de_dado restricoes
);
```

2.18.2 Restrições de Integridade (*Constraints*)

As restrições são regras aplicadas às colunas para garantir que os dados inseridos sejam válidos e consistentes. Sem elas, o banco de dados correria o risco de ter informações duplicadas ou incompletas.

As principais restrições são:

- `PRIMARY KEY`: Identifica unicamente cada registro. Não permite valores nulos nem repetidos.
- `NOT NULL`: Garante que a coluna não aceite valores vazios (campo obrigatório).
- `UNIQUE`: Garante que todos os valores na coluna sejam distintos (ex: CPF, e-mail).
- `DEFAULT`: Define um valor padrão caso nenhum seja informado na inserção.

2.18.3 Exemplo Comentado

Vamos analisar a criação da tabela `produto`, aplicando os conceitos aprendidos:

SQL

```sql
CREATE TABLE produto (
    id SERIAL PRIMARY KEY,            -- ID autoincrementado e chave
    ↪  primaria
    nome VARCHAR(100) NOT NULL,      -- Nome obrigatorio
    preco DECIMAL(10,2) NOT NULL,    -- Preco com 2 casas decimais
    estoque INTEGER DEFAULT 0,       -- Se nao informado, comeca com 0
    sku CHAR(8) UNIQUE               -- Codigo unico de produto
);
```

Dica do Professor: Planejamento

Antes de digitar o comando no terminal, sempre valide se os tipos de dados escolhidos suportam as necessidades do negócio. Mudar a estrutura de uma tabela (`ALTER TABLE`) depois que ela já possui milhões de registros é uma tarefa muito mais complexa e arriscada.

2.19 Inserção de Dados: O Comando INSERT INTO

Após criar a estrutura das tabelas com o CREATE TABLE, o próximo passo é popular o banco de dados. O comando INSERT é utilizado para inserir novos registros (linhas) em uma tabela existente.

2.19.1 Sintaxe Básica

Existem duas formas principais de utilizar este comando. A mais segura e recomendada é especificar explicitamente as colunas onde os dados serão inseridos:

SQL

```sql
INSERT INTO nome_da_tabela (coluna1, coluna2, ...)
VALUES (valor1, valor2, ...);
```

2.19.2 Regras Cruciais para Inserção

Para que a inserção ocorra sem erros, o desenvolvedor deve seguir algumas regras fundamentais de sintaxe e integridade:

- **Textos e Datas**: Devem sempre estar entre aspas simples (ex: 'João Silva', '2024-05-10').
- **Números**: Não utilizam aspas (ex: 1500.50, 25).
- **Ordem**: A ordem dos valores no VALUES deve ser exatamente a mesma das colunas listadas.
- **Campos Obrigatórios**: Colunas com a restrição NOT NULL devem obrigatoriamente receber um valor, a menos que possuam um DEFAULT.

2.19.3 Prática: Inserindo Dados na Tabela Pessoa

Vamos aplicar o comando à tabela que modelamos anteriormente. Observe que não precisamos inserir o campo ID, pois ele é do tipo SERIAL e o banco o gera automaticamente.

SQL

```sql
-- Insercao especificando as colunas
INSERT INTO pessoa (nome, cpf, email, data_nascimento)
VALUES ('Ana Souza', '12345678910', 'ana@email.com', '1992-08-24');

-- Insercao de multiplos registros em um unico comando
INSERT INTO pessoa (nome, cpf, email)
VALUES
    ('Carlos Luz', '98765432100', 'carlos@email.com'),
    ('Beatriz Vaz', '55544433322', 'beatriz@email.com');
```

Atenção: Erros de Restrição

Se você tentar inserir um CPF que já existe ou deixar de preencher um campo `NOT NULL`, o PostgreSQL impedirá a operação e retornará um erro de **violação de restrição**. Isso garante que o banco de dados nunca aceite dados inválidos ou "sujos".

Dica do Professor: Omissions

Campos que possuem `DEFAULT` (como o campo `ativo` ou `criado_em` da nossa tabela) podem ser omitidos no `INSERT`. O PostgreSQL preencherá automaticamente com o valor padrão definido na criação da tabela.

Capítulo 3

Normalização

A concepção de um banco de dados relacional eficiente vai muito além da simples identificação das entidades e de seus relacionamentos.

O modelo conceitual de banco de dados, como o representado pela notação de Peter-Chen, discutido em capítulos anteriores, e sua posterior transformação em um modelo lógico — composto por tabelas, colunas, chaves primárias e chaves estrangeiras — fornece apenas uma estrutura inicial para a organização dos dados. Todavia, essa estrutura ainda necessita de refinamento para evitar redundâncias, minimizar inconsistências e assegurar a integridade das informações armazenadas.[1] É nesse contexto que entra a **Normalização de Banco de Dados**.

O processo de normalização é uma técnica matemática formal, introduzida por Edgar F. Codd na década de 1970, no contexto do desenvolvimento do modelo relacional.[2] Posteriormente, o autor aprofundou e formalizou as formas normais em trabalhos subsequentes.[3]

Ele consiste na aplicação sistemática de um conjunto de regras visando organizar a estrutura das tabelas para minimizar a redundância de informações e evitar problemas indesejados nas operações de manipulação de dados, conhecidos como anomalias.

Para compreender a fundo a importância desse processo, este capítulo explorará inicialmente a natureza das anomalias de dados e os problemas intrínsecos aos esquemas lógicos mal estruturados. Uma vez estabelecido o diagnóstico, serão apresentados, de forma progressiva, os conceitos relativos às dependências funcionais e as diferentes Formas Normais, as quais fornecem os critérios estruturais indispensáveis para a consistência de um banco de dados relacional.

3.1 Anomalias de Dados

Conforme já mencionado, quando um banco de dados relacional não está devidamente normalizado, a redundância de informações em múltiplos locais não apenas desperdiça

[1] RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

[2] CODD, Edgar F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM**, Association for Computing Machinery, v. 13, n. 6, p. 377–387, 1970. DOI: 10.1145/362384.362685.

[3] CODD, Edgar F. **Further Normalization of the Data Base Relational Model**. San Jose, California, 1971.

espaço em disco — um problema crítico em décadas passadas — como também eleva o risco de inconsistências e contradições durante as manipulações de dados.

Nesse sentido, para compreender a real necessidade da normalização, é fundamental analisar os efeitos de estruturas mal desenhadas, as quais estão sujeitas a três tipos principais de anomalias: inserção, atualização e exclusão.[4]

Situação base para análise

Para compreender de forma concreta os efeitos das anomalias em bancos de dados não normalizados, considere a tabela fictícia de `Alunos_Cursos`, que busca armazenar, em uma única relação, informações relativas a alunos, cursos e departamentos. Essa estrutura representa um exemplo típico de modelagem inadequada, na qual diferentes entidades são indevidamente agrupadas, gerando redundância de dados e inconsistências potenciais.
A tabela a seguir será utilizada como base para a análise das anomalias de inserção, atualização e exclusão apresentadas nas subseções seguintes.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Departamento
101	Leandro	C10	Engenharia de Software	Faculdade de Tecnologia
102	Sergio	C10	Engenharia de Software	Faculdade de Tecnologia
103	Ronaldo	C20	Direito	Faculdade de Direito
104	Arthur	C10	Engenharia de Software	Faculdade de Tecnologia
105	Pedro	C30	Medicina	Faculdade de Medicina
106	Claudia	C40	Agronomia	Faculdade de Agronomia

Tabela 3.1: Tabela base para análise das anomalias.

3.1.1 Anomalia de Inserção

A anomalia de inserção ocorre quando não é possível inserir dados sobre uma determinada entidade sem inserir dados sobre outra entidade não relacionada no momento.[5]

Situação exemplo

Imagine que a universidade acaba de criar um novo curso chamado "Engenharia de Computação"(Código: C50, Departamento: Faculdade de Tecnologia).
Como não há, ainda, nenhum aluno matriculado nesse novo curso, não podemos registrar a existência do curso na Tabela `Alunos_Cursos`, a menos que criemos um "aluno fantasma". Isso ocorre porque fatos sobre cursos estão indevidamente acoplados a fatos sobre alunos.

[4]RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

[5]RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Departamento
....	...	...	...	....
105	Pedro	C30	Medicina	Faculdade de Medicina
106	Claudia	C40	Agronomia	Faculdade de Agronomia
107	NULL	C50	Engenharia de Computação	Faculdade de Tecnologia

Tabela 3.2: Exemplo de anomalia de inserção

3.1.2 Anomalia de Atualização

A anomalia de atualização ocorre quando, devido à duplicidade de informações, um dado armazenado em mais de um registro precisa ser alterado. Se apenas uma das ocorrências desse for atualizada, enquanto as demais permanecem inalteradas, o banco de dados passa a apresentar informações inconsistentes.[6]

Situação exemplo

Note que o nome do curso "Engenharia de Software"e seu departamento "Faculdade de Tecnologia"estão repetidos para os alunos Leandro, Sergio e Arthur. Se a universidade decidir alterar o nome do departamento de "Faculdade de Tecnologia"para "Faculdade de Engenharia", é preciso mudar o departamento nas três linhas.
Se, por falha do sistema ou falha de transação, apenas as duas primeiras linhas forem atualizadas, o banco de dados ficará num estado inconsistente, afirmando que Arthur cursa "Engenharia de Software"na "Faculdade de Engenharia"e Leandro cursa o mesmo curso com o mesmo `Cod_Curso` C10, mas em departamento diverso.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Departamento
101	Leandro	C10	Engenharia de Software	Faculdade de Engenharia
102	Sergio	C10	Engenharia de Software	Faculdade de Engenharia
103	Ronaldo	C20	Direito	Faculdade de Direito
104	Arthur	C10	Engenharia de Software	Faculdade de Tecnologia
....	...	...	...	....

Tabela 3.3: Exemplo de anomalia de atualização

3.1.3 Anomalia de Exclusão

A anomalia de exclusão ocorre quando a exclusão intencional de algumas informações de uma entidade leva também à perda não intencional de informações não relacionadas, ou seja, perde-se também informações informações de outra entidade.[7]

[6] RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

[7] RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

Situação exemplo

Imagine que o aluno "Pedro"(Matrícula 105) decida trancar sua matrícula ou abandonar o curso, e seu registro precise ser excluído do sistema. Ao excluir o aluno 105, perderemos não apenas as informações do Pedro, mas também perderemos completamente a informação de que a universidade oferece o curso C30 (Medicina) e que este pertence à Faculdade de Medicina.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Departamento
....	...	...	...	....
104	Arthur	C10	Engenharia de Software	Faculdade de Tecnologia
105	Pedro	C30	Medicina	Faculdade de Medicina
106	Claudia	C40	Agronomia	Faculdade de Agronomia

Tabela 3.4: Exemplo de anomalia de exclusão.

3.2 Dependências Funcionais

Uma relação com redundância pode ser normalizada, resolvendo as anomalias supramencionadas, por sua decomposição ou sua substituição por relações menores que contêm as mesmas informações, mas sem redundância.[8] Mas para fazer a decomposição/substituição das relações de forma a normalizá-las, é necessário dominar o conceito de Dependência Funcional (DF).

Uma dependência funcional ocorre quando um conjunto de atributos determina outro conjunto de atributos dentro de uma relação (tabela).

Formalmente, diz-se que um atributo, ou conjunto de atributos, **Y** é funcionalmente dependente de um atributo, ou conjunto de atributos, **X**, se, para cada valor de **X** estiver associado a um único valor de **Y**.[9]

Neste contexto, X é chamado de atributo determinante e Y é o atributo dependente e podemos representar da seguinte forma:

$$X \rightarrow Y$$

(lido como "X determina Y")

Deve-se ter cuidado, a dependência funcional determina que para cada valor de X temos sempre o mesmo valor de Y, mas não impede que muitos valores distintos de X podem ter o mesmo valor de Y.

Situação exemplo

Considere o cadastro de alunos apresentado anteriormente, no qual cada matrícula identifica um único aluno. Nesse caso, temos que a matrícula determina funcionalmente o nome do aluno:

[8] RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

[9] DATE, C. J. **Introdução a Sistemas de Banco de Dados**. Tradução: Daniel Vieira. Rio de Janeiro: Elsevier, 2003. Tradução de: Introduction to Database Systems, 8th ed. ISBN 978-85-352-8445-4.

$$Matricula \rightarrow Nome_Aluno$$

Isso significa que, conhecendo a matrícula, sempre é possível identificar um único nome associado a ela. Por outro lado, o inverso não é necessariamente verdadeiro, dois ou mais alunos, com matrículas distintas, podem possuir o mesmo nome. A tabela abaixo ilustra essa situação.

Matricula	**Nome_Aluno**
101	Leandro
102	Sergio
...	...
107	Leandro

De forma prática, na teoria relacional, a chave primária de uma tabela deve determinar todos os outros atributos dessa tabela. O problema surge quando os atributos não-chave começam a determinar outros atributos não-chave, ou quando são determinados apenas uma parte da chave primária. Vejamos os tipos de dependências funcionais, nas próximas subseções.

3.2.1 Dependência Funcional Transitiva

Ocorre quando um atributo não-chave de uma tabela, depende de outro atributo não-chave da mesma tabela, que por sua vez depende do atributo chave da tabela.[10]

Formalmente, se temos uma chave primária X, e atributos não-chave Y e Z. Se a chave determina Y e Y determina Z , então Z depende da chave X de forma *transitiva* mediante Y. Note que, o atributo Z não tem relação direta com a chave X, mas sim com outro atributo da tabela Y.[11]

Em forma de notação, temos que se: $X \rightarrow Y$ e $Y \rightarrow Z$, então, podemos concluir que $X \rightarrow Z$.

Situação exemplo

Retornando à tabela inicial `Alunos_Cursos` (Tabela 3.1), considere que cada aluno pode estar matriculado em apenas um curso, e cada curso pertence a um departamento. Nesse caso, a chave primária simples é a `Matricula`.

Matricula	**Nome_Aluno**	**Cod_Curso**	**Nome_Curso**	**Departamento**
101	Leandro	C10	Engenharia de Software	Faculdade de Tecnologia
102	Sergio	C10	Engenharia de Software	Faculdade de Tecnologia
103	Ronaldo	C20	Direito	Faculdade de Direito
105	Pedro	C30	Medicina	Faculdade de Medicina

[10] ANGELOTTI, Elaini Simoni. **Banco de dados**. Curitiba: Editora do Livro Técnico, 2010. p. 120. ISBN 978-85-63687-02-9.

[11] DATE, C. J. **Introdução a Sistemas de Banco de Dados**. Tradução: Daniel Vieira. Rio de Janeiro: Elsevier, 2003. Tradução de: Introduction to Database Systems, 8th ed. ISBN 978-85-352-8445-4.

Neste cenário, sabemos que a chave primária determina todos os outros atributos da tabela. Logo:

$$Matricula \rightarrow Cod_Curso$$

Porém, os atributos relacionados ao curso (`Nome_Curso` e `Departamento`) dependem unicamente do `Cod_Curso`, e não diretamente da matrícula do aluno em si. Portanto, temos a seguinte dependência entre atributos não-chave:

$$Cod_Curso \rightarrow Departamento$$

Como a matrícula determina o código do curso, e o código do curso determina o departamento, concluímos que o `Departamento` depende **transitivamente** da chave primária `Matricula`. O mesmo ocorre com o `Nome_Curso`.

3.2.2 Dependência Funcional Total e Dependência Funcional Parcial

A análise de dependência funcional total ou parcial faz-se necessária quando a tabela possui uma chave primária composta, ou seja, aquela formada por mais de um atributo. A dependência funcional total se dá quando o atributo não-chave Y depende funcionamemente de toda a chave primária composta {X, Z}.[12]

$$\{X, Y\} \rightarrow Z$$

De forma análoga, a dependência funcional parcial se dá quando o atributo atributo não-chave Y depende funcionamemente de apenas uma parte da chave primária composta {X, Z}.

$$X \rightarrow Y$$

Situação exemplo

Considere uma ampliação do cenário da universidade, em que a tabela agora registra a nota final obtida (média) pelo aluno no curso. Nesta nova versão da tabela, a chave primária deve composta por `Matricula` e `Cod_Curso`, afinal um aluno pode realizar vários cursos e cada curso pode possuir vários alunos.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Nota
101	Leandro	C10	Engenharia de Software	9.0
101	Leandro	C20	Direito	8.5
102	Sergio	C10	Engenharia de Software	7.5
103	Ronaldo	C30	Medicina	8.0

Nesse caso, em relação à nota, temos uma dependência funcional total, sendo necessário conhecer ambos os atributos da chave composta (aluno e curso) para

[12] ANGELOTTI, Elaini Simoni. **Banco de dados**. Curitiba: Editora do Livro Técnico, 2010. p. 120. ISBN 978-85-63687-02-9.

determiná-la.

$$\{\text{Matricula}, \text{Cod_Curso}\} \rightarrow Nota$$

Entretanto, também observamos as seguintes dependências:

$$Matricula \rightarrow Nome_Aluno$$
$$Cod_Curso \rightarrow Nome_Curso$$

Essas são dependências funcionais parciais, nas quais os atributos `Nome_Aluno` e `Nome_Curso` são determinados por apenas parte da chave primária composta.

3.2.3 Dependência Funcional Trivial e Não Trivial

Uma dependência funcional é classificada como trivial quando o conjunto de atributos dependentes constitui um subconjunto do conjunto de atributos determinantes. Em outras palavras, o lado direito da dependência contém apenas atributos que já aparecem no lado esquerdo.[13]

Em notação funcional, se $\{X, Y\}$ é chave de uma determinada relação R)e ocorre $\{X, Y\} \rightarrow Y$, então essa dependência funcional é considerada trivial.

Dependências triviais não representam conhecimento novo sobre os dados, mas precisam ser compreendidas porque aparecem formalmente nas definições das formas normais mais avançadas. De forma simples, se os valores de um conjunto de atributos já são conhecidos, conhecer o valor de apenas parte desse conjunto não acrescenta nenhuma nova informação.

Por outro lado, uma dependência funcional é não trivial quando, para a mesma relação R, um determinado atributo dependente Z não é um subconjunto da chave determinante {X, Y}. Em outras palavras, o lado direito da dependência apresenta um atributo que não está contido no lado esquerdo.

Nesse caso, a dependência funcional expressa uma relação real entre os atributos da tabela, indicando que o valor de um atributo pode ser determinado a partir de um conjunto de outros atributos.

São justamente as dependências funcionais não triviais que possuem maior relevância no processo de normalização, pois revelam possíveis redundâncias e relações estruturais entre os dados. A identificação dessas dependências permite decompor tabelas de forma adequada, eliminando inconsistências e conduzindo o esquema relacional às diferentes formas normais.

Situação exemplo

Considere mais uma vez a tabela que registra a nota final obtida pelo aluno em um curso. A chave primária é a composição de `Matricula` e `Cod_Curso`.

[13] DATE, C. J. **Introdução a Sistemas de Banco de Dados**. Tradução: Daniel Vieira. Rio de Janeiro: Elsevier, 2003. Tradução de: Introduction to Database Systems, 8th ed. ISBN 978-85-352-8445-4.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Nota
101	Leandro	C10	Engenharia de Software	9.0
101	Leandro	C20	Direito	8.5
102	Sergio	C10	Engenharia de Software	7.5
103	Ronaldo	C30	Medicina	8.0

Neste caso, podemos observar as seguintes dependências funcionais **triviais**:

$$\{\text{Matricula}, \text{Cod_Curso}\} \rightarrow \text{Matricula}$$

$$\{\text{Matricula}, \text{Cod_Curso}\} \rightarrow \text{Cod_Curso}$$

É evidente que se possuímos os valores de `Matricula` e `Cod_Curso`, seremos capazes de determinar unicamente o valor de `Matricula` (ou do próprio `Cod_Curso`). Trata-se de uma constatação lógica e natural do modelo.
Entretanto, a dependência abaixo é considerada **não trivial**, pois o atributo `Nota` não está contido na chave primária e traz, de fato, uma nova informação sobre a relação:

$$\{\text{Matricula}, \text{Cod_Curso}\} \rightarrow \text{Nota}$$

3.3 O Processo de Normalização e as Formas Normais mais utilizadas

O processo de normalização consiste em um conjunto de técnicas aplicadas ao projeto de bancos de dados relacionais com o objetivo de organizar os dados de forma adequada, reduzir redundâncias e evitar anomalias.

Esse processo baseia-se na análise das dependências existentes entre os atributos de uma relação, especialmente nas dependências funcionais, buscando estruturar o esquema relacional de maneira mais consistente e eficiente.

A normalização é realizada por meio da aplicação sucessiva de critérios estruturais conhecidos como *formas normais*.

Conceitualmente, uma forma normal pode ser entendida como uma condição ou conjunto de restrições estruturais aplicadas a uma relação, cujo objetivo é reduzir redundâncias e evitar anomalias.[14]

Conforme apontam as principais literaturas da área, no projeto de bancos de dados comerciais busca-se garantir que o modelo atinja, pelo menos, a Terceira Forma Normal (3FN). O processo de normalização é gradual: não é possível que uma relação esteja na 2FN sem antes satisfazer os requisitos da 1FN, e assim sucessivamente.

Assim, o fato de uma tabela encontrar-se em uma forma normal mais avançada implica, necessariamente, o cumprimento cumulativo de todas as formas normais precedentes. Uma relação que esteja, por exemplo, na Terceira Forma Normal (3FN) já atende integralmente aos requisitos da Primeira Forma Normal (1FN) e da Segunda Forma Normal (2FN). Tal lógica revela o caráter hierárquico e progressivo do processo de normalização,

[14] CODD, Edgar F. **Further Normalization of the Data Base Relational Model**. San Jose, California, 1971.

no qual cada etapa sucessiva incorpora e refina as restrições estruturais das anteriores, promovendo um modelo de dados mais consistente, com menor incidência de anomalias.

Vamos conduzir um Estudo de Caso completo, ampliando a tabela tabela fictícia de `Alunos_Cursos`. Imagine que a secretaria acadêmica de uma instituição de ensino apresentou a seguinte "Ficha de Histórico Escolar"manual em formato de planilha e nos pediu para transformá-la em um banco de dados relacional (Tabela 3.5).

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Departamento	Cod_Disc	Nome_Disciplina	Nota
101	Leandro	C10	Eng. de Software	Fac. Tecnologia	D01	Banco de Dados	9.0
					D02	Prog. Orientada a Objetos	8.5
					D03	Redes de Computadores	7.0
102	Sergio	C10	Eng. de Software	Fac. Tecnologia	D01	Banco de Dados	8.0
					D04	Eng. de Requisitos	9.5
103	Ronaldo	C20	Direito	Fac. Direito	D05	Direito Civil	8.0

Tabela 3.5: Tabela em forma não normalizada, com grupos repetitivos.

Cabe ressaltar que a Primeira Forma Normal (1FN), a Segunda Forma Normal (2FN) e a Terceira Forma Normal (3FN), apresentadas nas subseções seguintes, têm como base conceitual o trabalho clássico de Edgar F. Codd, nos artigos *"A Relational Model of Data for Large Shared Data Banks"* e *"Further Normalization of the Data Base Relational Model"*, nos quais são discutidos os princípios fundamentais da normalização em bancos de dados relacionais.[15]

Esses conceitos constituem o alicerce teórico do processo de normalização e são amplamente reconhecidos na literatura da área de Banco de Dados, servindo como referência para a organização adequada de esquemas relacionais, com o objetivo de reduzir redundâncias e evitar anomalias de inserção, atualização e exclusão.

Destaca-se que, ao longo das próximas subseções, diversas definições derivam diretamente desses fundamentos teóricos. Entretanto, optou-se por não inserir citações bibliográficas em todos os parágrafos em que tais ideias aparecem. Essa decisão foi adotada exclusivamente por razões de clareza textual e fluidez da leitura, evitando a repetição excessiva de referências em trechos consecutivos, sem prejuízo do devido reconhecimento da obra original que fundamenta a discussão apresentada.

3.3.1 Forma Não Normalizada (FNN ou 0FN)

A Tabela 3.5 representa a tabela de origem do usuário, não normalizada. Ela mistura em um único plano de visão dados pertencentes às entidades: Aluno, Curso e Disciplina. Como o histórico de um aluno contém múltiplas disciplinas cursadas, existe na tabela o que chamamos de grupos repetitivos de dados, ou também conhecidos como tabelas aninhadas. Observe, por exemplo, que na linha da matrícula 101 os dados do aluno (Matrícula, Nome, Curso, Departamento) aparecem apenas uma vez, enquanto existem múltiplas informações nas colunas de disciplinas (D01, D02, D03).

[15] CODD, Edgar F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM**, Association for Computing Machinery, v. 13, n. 6, p. 377–387, 1970. DOI: 10.1145/362384.362685; CODD, Edgar F. **Further Normalization of the Data Base Relational Model**. San Jose, California, 1971.

3.3.2 Primeira Forma Normal (1FN)

Bancos de Dados Relacionais exigem que as tabelas sejam bidimensionais e estruturadas de forma plana, sem a presença de listas ou múltiplos valores armazenados em uma mesma célula.

Assim, uma tabela (relação) encontra-se na Primeira Forma Normal (1FN) se, e somente se, todos os seus atributos contêm apenas valores atômicos, ou seja, não podem existir atributos compostos, atributos multivalorados ou grupos repetitivos de dados.

De forma prática, para adequar a tabela à Primeira Forma Normal, deve-se, inicialmente, decompor eventuais atributos compostos em atributos simples, garantindo que cada coluna represente uma única informação indivisível.

No que se refere aos atributos multivalorados e aos grupos repetitivos, uma abordagem possível para eliminação se dá pela criação de tuplas distintas para cada valor. Para isso, os valores repetidos são deslocados para novas linhas, repetindo-se os demais atributos necessários, de modo que cada registro contenha apenas um único valor por coluna, preservando a correta associação entre os dados.

Situação exemplo

Aplicando-se os conceitos da Primeira Forma Normal ao exemplo anteriormente apresentado, obtém-se uma nova estrutura relacional na qual todos os atributos passam a conter valores atômicos e os grupos repetitivos são eliminados. Para isso, cada disciplina cursada por um aluno passa a ser representada em uma linha distinta, garantindo que não existam múltiplos valores em uma mesma célula.

Matricula	Nome_Aluno	Cod_Curso	Nome_Curso	Departamento	Cod_Disc	Nome_Disciplina	Nota
101	Leandro	C10	Eng. de Software	Fac. Tecnologia	D01	Banco de Dados	9.0
101	Leandro	C10	Eng. de Software	Fac. Tecnologia	D02	Prog. Orientada a Objetos	8.5
101	Leandro	C10	Eng. de Software	Fac. Tecnologia	D03	Redes de Computadores	7.0
102	Sergio	C10	Eng. de Software	Fac. Tecnologia	D01	Banco de Dados	8.0
102	Sergio	C10	Eng. de Software	Fac. Tecnologia	D04	Eng. de Requisitos	9.5
103	Ronaldo	C20	Direito	Fac. Direito	D05	Direito Civil	8.0

Tabela 3.6: Tabela em 1FN - Remoção dos grupos repetitivos.

Observa-se que a tabela passa a apresentar uma estrutura tipicamente relacional, na qual cada linha representa uma ocorrência única da relação entre aluno, curso e disciplina. Entretanto, essa reorganização impõe uma consequência relevante do ponto de vista da modelagem: a redefinição da chave primária.

Nota-se que o atributo `Matricula`, isoladamente, não é suficiente para identificar univocamente os registros, uma vez que um mesmo aluno pode estar associado a múltiplas disciplinas. De igual modo, o atributo `Cod_Disc` também não é capaz de identificar uma tupla de forma única, pois uma mesma disciplina pode ser cursada por diferentes alunos.

Dessa forma, torna-se necessário adotar uma chave primária composta, formada pela combinação dos atributos (`Matricula`, `Cod_Disc`), a qual, em conjunto, garante a unicidade de cada registro na relação.

Outro problema é que ao utilizar esta abordagem para adequar a tabela à 1FN, resolve-se a estrutura tabular, mas invariavelmente gera redundâncias absurdas. Note que para armazenar as três disciplinas do aluno 101, o sistema registrou o

seu nome "Leandro"e o seu curso três vezes. Isso causa anomalias de atualização iminentes, exatamente como discutido nas seções anteriores.

Uma segunda abordagem, mais adequada sob a ótica do projeto lógico de bancos de dados, consiste em extrair os atributos multivalorados e grupos repetitivos para relações próprias, ao invés de simplesmente replicar dados em múltiplas tuplas.[16]

Nessa perspectiva, os dados são reorganizados em tabelas distintas, cada uma representando uma entidade ou relacionamento específico, eliminando-se a necessidade de repetição da chave principal com atributos redundantes. Assim, os atributos que se repetem deixam de estar aninhados em uma única tabela e passam a compor uma nova relação, vinculada à entidade principal por meio de chave estrangeira.

Essa abordagem não apenas atende à 1FN, ao eliminar atributos multivalorados, como também evita a redundância excessiva observada na solução anterior. Com isso, reduz-se significativamente o risco de anomalias de atualização, inserção e exclusão, promovendo maior integridade e consistência dos dados, além de alinhar o modelo às boas práticas de normalização relacional.

Situação exemplo

Aplicando-se essa segunda abordagem ao exemplo em análise, observa-se uma mudança estrutural significativa no modelo de dados.
A seguir, apresenta-se a nova organização dos dados, agora distribuídos em duas relações distintas. A primeira representa a entidade principal (aluno e curso), enquanto a segunda armazena as informações anteriormente repetidas, relativas às disciplinas cursadas.

Matricula	**Nome_Aluno**	**Cod_Curso**	**Nome_Curso**	**Departamento**
101	Leandro	C10	Eng. de Software	Fac. Tecnologia
102	Sergio	C10	Eng. de Software	Fac. Tecnologia
103	Ronaldo	C20	Direito	Fac. Direito

Tabela 3.7: Tabela principal (ALUNO_CURSO) após a separação do grupo repetitivo.

Matricula (FK)	**Cod_Disc**	**Nome_Disciplina**	**Nota**
101	D01	Banco de Dados	9.0
101	D02	Programação Orientada a Objetos	8.5
101	D03	Redes de Computadores	7.0
102	D01	Banco de Dados	8.0
102	D04	Engenharia de Requisitos	9.5
103	D05	Direito Civil	8.0

[16] ANGELOTTI, Elaini Simoni. **Banco de dados**. Curitiba: Editora do Livro Técnico, 2010. p. 120. ISBN 978-85-63687-02-9.

Tabela 3.8: Tabela do grupo repetitivo extraído (ALUNO_DISCIPLINA).

Essa reorganização evidencia uma evolução importante no processo de normalização, cada informação passa a ser armazenada apenas no contexto em que possui significado próprio, evitando duplicações desnecessárias.

Por fim, observa-se que a associação entre as tabelas é preservada por meio do atributo `Matricula`, que atua como elo lógico entre as relações, caracterizando uma estrutura típica de chave primária e chave estrangeira.

Observação: os próximos exemplos serão baseados nessa estrutura de tabelas, com a aplicação da segunda abordagem aqui apresentada.

Nessa segunda abordagem, a eliminação dos grupos repetitivos não ocorre pela simples replicação dos dados do aluno em diversas linhas, mas sim pela separação lógica das entidades e de seus relacionamentos. Com isso, evita-se a repetição desnecessária de informações textuais, como o nome do aluno, o nome do curso e o nome da disciplina, tornando a estrutura mais próxima de um esquema relacional adequadamente projetado.

A separação das relações permite maior controle sobre as dependências funcionais existentes, facilitando futuras etapas do processo de normalização, como a eliminação de dependências parciais e transitivas. Com isso, o modelo torna-se mais flexível, consistente e aderente aos princípios do modelo relacional.

Cumpre destacar, contudo, que essa segunda abordagem ultrapassa a simples eliminação de grupos repetitivos, pois já promove uma decomposição lógica das entidades e relacionamentos envolvidos. Em verdade, a separação das relações permite maior controle sobre as dependências funcionais existentes, facilitando futuras etapas do processo de normalização, como a eliminação de dependências parciais e transitivas. Com isso, o modelo torna-se mais flexível, consistente e aderente aos princípios do modelo relacional.

3.3.3 Segunda Forma Normal (2FN)

Uma tabela (relação) encontra-se na Segunda Forma Normal (2FN) se, e somente se, estiver previamente na Primeira Forma Normal (1FN) e não apresentar dependências funcionais parciais. Em outras palavras, além de possuir apenas atributos atômicos, todo atributo que não integra a chave primária deve depender funcionalmente da chave primária em sua totalidade, e não apenas de parte dela.

Importante destacar que a Segunda Forma Normal constitui uma preocupação relevante apenas em tabelas cuja chave primária é composta. Isso porque, quando a chave primária é simples, não há possibilidade de ocorrência de dependência parcial; assim, uma vez atendidos os requisitos da 1FN, a tabela estará automaticamente em 2FN.

Do ponto de vista prático, a adequação à Segunda Forma Normal exige a decomposição da relação original, de modo a eliminar as dependências parciais identificadas. Para tanto, os atributos que dependem apenas de parte da chave primária devem ser extraídos para novas tabelas, nas quais essa parte da chave passa a atuar como chave primária. Por sua vez, os atributos que dependem integralmente da chave composta permanecem na relação original.[17]

[17] ANGELOTTI, Elaini Simoni. **Banco de dados**. Curitiba: Editora do Livro Técnico, 2010. p. 120. ISBN 978-85-63687-02-9.

Situação exemplo

Considerando a tabela relacionamento Aluno_Disciplina obtida na Primeira Forma Normal (Tabela 3.8), cuja chave primária é composta por (`Matricula`, `Cod_Disc`), procede-se à análise das dependências funcionais, a fim de verificar sua adequação à Segunda Forma Normal:

- O atributo `Nome_Disciplina` não depende da chave composta em sua totalidade, sendo determinado exclusivamente pelo `Cod_Disc`, independentemente da matrícula do aluno. Tal situação caracteriza uma **dependência funcional parcial**.
- O atributo **Nota**, por sua vez, depende integralmente da combinação (`Matricula`, `Cod_Disc`), uma vez que não pode ser determinado isoladamente nem pelo aluno, nem pela disciplina. Trata-se, portanto, de uma **dependência funcional total**.

Diante dessa análise, verifica-se que a relação ainda apresenta dependência funcional parcial, o que impede seu enquadramento na Segunda Forma Normal.
Para eliminar essa inconsistência estrutural, torna-se necessária a decomposição da tabela, isolando-se os atributos conforme seus respectivos determinantes funcionais.

Matricula	**Nome_Aluno**	**Cod_Curso**	**Nome_Curso**	**Departamento**
101	Leandro	C10	Eng. de Software	Fac. Tecnologia
102	Sergio	C10	Eng. de Software	Fac. Tecnologia
103	Ronaldo	C20	Direito	Fac. Direito

Tabela 3.9: Tabela ALUNO_CURSO repetida do exemplo anterior

Matricula (FK)	**Cod_Disc (FK)**	**Nota**
101	D01	9.0
101	D02	8.5
101	D03	7.0
102	D01	8.0
102	D04	9.5
103	D05	8.0

Tabela 3.10: Tabela de relacionamento ALUNO_DISCIPLINA agora somente com a nota

Cod_Disc	Nome_Disciplina
D01	Banco de Dados
D02	Programação Orientada a Objetos
D03	Redes de Computadores
D04	Engenharia de Requisitos
D05	Direito Civil

Tabela 3.11: Tabela DISCIPLINA criada pela deconposição de eliminar a dependência parcial.

Com a decomposição realizada, elimina-se a dependência funcional parcial anteriormente identificada. A tabela de relacionamento passa a armazenar exclusivamente atributos que dependem integralmente da chave composta, enquanto o atributo `Nome_Disciplina` é corretamente alocado em uma relação própria, determinada por `Cod_Disc`.

Dessa forma, o modelo passa a atender aos requisitos da Segunda Forma Normal, reduzindo redundâncias e promovendo maior consistência estrutural dos dados.

Contudo, embora a estrutura esteja agora adequada à 2FN, ainda subsistem dependências funcionais transitivas, especialmente entre `Cod_Curso`, `Nome_Curso` e `Departamento`, o que será objeto de análise na próxima etapa do processo de normalização.

3.3.4 Terceira Forma Normal (3FN)

A **Terceira Forma Normal (3FN)** exige que a tabela esteja conformizada com a 2FN e que nenhum de seus atributos não haja dependência transitiva.

Assim, o atributo deve estar logicamente vinculado à chave, não podendo depender apenas de parte dela (o que caracterizaria dependência parcial) nem de outro atributo não-chave que por sua vez depende da chave (o que configuraria dependência transitiva).

Do ponto de vista prático, a eliminação de dependências transitivas exige a decomposição da relação original, de modo a isolar, em uma nova entidade, os atributos que não dependem diretamente da chave primária. Nessa abordagem, tais atributos passam a compor uma relação própria, cuja chave primária corresponde ao seu determinante natural, enquanto na relação original mantém-se apenas a referência a essa nova entidade, por meio de chave estrangeira. Com isso, assegura-se que todos os atributos remanescentes dependam exclusivamente da chave primária da relação, atendendo aos requisitos da Terceira Forma Normal.[18]

A observância da 3FN assegura que cada atributo represente uma informação genuína sobre a entidade modelada, evitando redundâncias e garantindo maior consistência estrutural ao esquema relacional.

[18] ANGELOTTI, Elaini Simoni. **Banco de dados**. Curitiba: Editora do Livro Técnico, 2010. p. 120. ISBN 978-85-63687-02-9.

Situação exemplo

Considerando a relação `ALUNO_CURSO`, obtida na etapa da Segunda Forma Normal (Tabela 3.9), cuja chave primária é a `Matricula`, procede-se à análise das dependências funcionais, a fim de verificar sua adequação à Terceira Forma Normal:

- O atributo `Nome_Aluno` depende diretamente da `Matricula`, que é a chave primária da relação. Trata-se, portanto, de uma dependência funcional adequada.
- O atributo `Cod_Curso` também depende diretamente da `Matricula`, uma vez que identifica o curso ao qual o aluno está vinculado. Também se trata de dependência funcional adequada.
- O atributo `Nome_Curso`, contudo, não depende diretamente da `Matricula`, mas sim do `Cod_Curso`. Verifica-se, assim, a existência de **dependência funcional transitiva**, pois a chave primária determina `Cod_Curso`, e este, por sua vez, determina `Nome_Curso`.
- O mesmo ocorre com o atributo `Departamento`, que igualmente não depende diretamente da `Matricula`, mas do `Cod_Curso`, configurando outra **dependência funcional transitiva**.

Diante dessa análise, verifica-se que a relação `ALUNO_CURSO` ainda não atende aos requisitos da Terceira Forma Normal, pois contém atributos não-chave que dependem de outro atributo não-chave.
Para eliminar essa inconsistência estrutural, torna-se necessária nova decomposição, isolando-se em uma relação própria os atributos que dependem funcionalmente de `Cod_Curso`.

Cod_Curso	**Nome_Curso**	**Departamento**
C10	Eng. de Software	Fac. Tecnologia
C20	Direito	Fac. Direito

Tabela 3.12: Tabela CURSO criada para eliminar a dependência transitiva.

Matricula	**Nome_Aluno**	**Cod_Curso** (FK)
101	Leandro	C10
102	Sergio	C10
103	Ronaldo	C20

Tabela 3.13: Tabela ALUNO após a decomposição da relação ALUNO_CURSO.

Com a decomposição realizada, eliminam-se as dependências transitivas anteriormente identificadas na relação `ALUNO_CURSO`. Os atributos `Nome_Curso` e

`Departamento`, que dependiam funcionalmente de `Cod_Curso`, passam a ser armazenados na relação `CURSO`, cuja chave primária é o próprio `Cod_Curso`.

Por sua vez, a relação `ALUNO` passa a conter apenas atributos que dependem diretamente de sua chave primária, mantendo `Cod_Curso` apenas como chave estrangeira.

As relações `DISCIPLINA` e `ALUNO_DISCIPLINA`, obtidas na etapa da Segunda Forma Normal, permanecem inalteradas, pois já atendem aos requisitos da Terceira Forma Normal.

Dessa forma, o modelo passa a atender plenamente à 3FN, assegurando que todo atributo não-chave dependa exclusivamente da chave primária, sem dependências parciais ou transitivas.

3.4 Formas Normais Avançadas

As formas normais superiores à Terceira Forma Normal (3FN) são, em geral, menos frequentes na prática cotidiana de modelagem de dados, mas assumem relevância em cenários mais complexos, especialmente em contextos acadêmicos ou em sistemas que envolvem múltiplas variáveis independentes associadas a uma mesma chave, bem como em estruturas que apresentam múltiplas chaves candidatas.

Destaca-se que, ao longo das próximas subseções, diversas definições e construções conceituais derivam diretamente dos fundamentos teóricos apresentados por C. J. Date[19], sendo também os exemplos aqui utilizados adaptações do trabalho do referido autor. Contudo, optou-se por não inserir citações bibliográficas diretamente nos parágrafos em que tais ideias se manifestam, por razões de clareza textual e fluidez da leitura, evitando a repetição excessiva de referências sem prejuízo do devido reconhecimento da obra original.

umpre registrar que a abordagem adotada no presente texto envolve determinadas simplificações metodológicas, voltadas à organização e exposição progressiva dos conceitos, principalmente com o foco em facilitar a compreenção de analistas iniciais. Tais simplificações não exaurem o tema, razão pela qual se recomenda a consulta direta à obra do autor mencionado, cuja profundidade e rigor teórico são essenciais para o completo domínio da matéria.

3.4.1 Forma Normal de Boyce-Codd (BCNF)

Conforme explica Christopher J. Date[20] definição original da Terceira Forma Normal, proposta por Codd, mostrou-se insuficiente para lidar adequadamente com situações mais complexas, especialmente quando uma relação possui múltiplas chaves candidatas compostas que compartilham atributos entre si. Nesses casos, a formulação inicial da 3FN não conseguia evitar determinadas anomalias estruturais.

Diante dessa limitação, passou-se a buscar uma definição mais rigorosa da 3FN, capaz de tratar adequadamente dependências funcionais que escapavam à formulação original.

[19] DATE, C. J. **Introdução a Sistemas de Banco de Dados**. Tradução: Daniel Vieira. Rio de Janeiro: Elsevier, 2003. Tradução de: Introduction to Database Systems, 8th ed. ISBN 978-85-352-8445-4.

[20] DATE, C. J. **Introdução a Sistemas de Banco de Dados**. Tradução: Daniel Vieira. Rio de Janeiro: Elsevier, 2003. Tradução de: Introduction to Database Systems, 8th ed. ISBN 978-85-352-8445-4.

Nesse contexto, destaca-se que a primeira formalização relevante do problema foi apresentada por Heath[21], ao identificar anomalias decorrentes de determinadas dependências funcionais em esquemas relacionais. Posteriormente, Codd[22] aprofundou a análise teórica dessas limitações, consolidando uma definição mais restritiva no âmbito do modelo relacional.

Assim, essa nova formulação, por impor uma condição mais rigorosa do que a 3FN originalmente proposta, não foi tratada como mera redefinição, mas sim como uma nova forma normal, denominada Forma Normal de Boyce-Codd (FNBC), em reconhecimento à contribuição de Boyce e Codd na sua sistematização.

Importa destacar que, na prática, os cenários que motivaram essa reformulação não são tão frequentes. Assim, em muitas situações, a Terceira Forma Normal e a Forma Normal de Boyce-Codd produzem resultados equivalentes. Entretanto, quando há múltiplas chaves candidatas com sobreposição, a FNBC se mostra necessária para garantir a eliminação completa de redundâncias e inconsistências.

Uma tabela encontra-se na FNBC se, e somente se, para todas as suas dependências funcionais não triviais $X \to Y$, X for uma superchave.[23]

Conforme explica William Kent[24], nessa forma normal todo atributo não-chave deve fornecer um fato sobre a chave, toda a chave e nada mais além da chave. Nesse contexto, o termo "fato" deve ser compreendido como uma informação que descreve diretamente a entidade identificada pela chave primária, ou seja, uma propriedade que lhe é intrínseca e que dela depende exclusivamente.

Em termos simples, nenhum atributo pode ser determinado por um atributo não-chave e para adequar a relação à FNBC, impõe-se sua decomposição em duas tabelas, de modo que cada dependência funcional passe a ocorrer em um contexto no qual seu determinante seja efetivamente uma chave.

Situação exemplo

Considere o seguinte cenário: um aluno pode se inscrever em múltiplos Cursos Extracurriculares. Para cada curso em que está matriculado, ele terá aula com apenas um Professor específico. Contudo, um mesmo curso pode ser ministrado por diferentes professores ao longo do tempo. A estrutura relacional que representa essas inscrições pode ser definida como: `(Aluno, Curso_Extra, Professor)`

[21] HEATH, I. J. Unacceptable File Operations in a Relational Database. *In:* PROCEEDINGS of the 1971 ACM SIGFIDET Workshop on Data Description, Access, and Control. San Diego, California: [*s. n.*], nov. 1971.

[22] CODD, E. F. Recent Investigations into Relational Data Base Systems. *In:* PROCEEDINGS of the IFIP Congress 1974. [*S. l.: s. n.*], 1974. p. 1017–1021.

[23] RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.

[24] KENT, William. A Simple Guide to Five Normal Forms in Relational Database Theory. **Communications of the ACM**, v. 26, n. 2, p. 120–125, fev. 1983. DOI: 10.1145/358061.358064.

Aluno	**Curso_Extra**	**Professor**
Ana	Teatro	Carlos
Ana	Música	Fernanda
Bruno	Teatro	Carlos
Bruno	Música	Fernanda
Carla	Teatro	Carlos

Tabela 3.14: Tabela de inscrições em cursos extracurriculares.

Observa-se que o atributo `Professor` depende da totalidade da chave primária (`Aluno`, `Curso_Extra`), não havendo dependência parcial nem dependência transitiva em relação a atributos não-chave. Assim, a relação atende, do ponto de vista formal, aos requisitos da Terceira Forma Normal.

$$(\texttt{Aluno}, \texttt{Curso_Extra}) \rightarrow \texttt{Professor}$$

Todavia, verifica-se uma inconsistência estrutural mais sutil: o atributo não-chave `Professor` determina parte da chave primária (`Curso_Extra`). Isso configura uma violação da Forma Normal de Boyce-Codd, pois existe uma dependência funcional na qual o determinante não é superchave.

$$\texttt{Professor} \rightarrow \texttt{Curso_Extra}$$

Como consequência da segunda dependência descrita acima, o modelo permanece sujeito a anomalias. Caso um professor passe a lecionar outro curso extracurricular, será necessário atualizar múltiplas tuplas da relação, repetindo a informação para todos os alunos vinculados a esse professor, o que caracteriza uma típica anomalia de atualização

Para eliminar essa inconsistência, torna-se necessária a decomposição da relação original em duas novas relações:

Professor	**Curso_Extra**
Carlos	Teatro
Fernanda	Música

Tabela 3.15: Tabela PROFESSOR_CURSO_EXTRA, criada para representar a dependência `Professor` $\rightarrow$ `Curso_Extra`.

Aluno	**Professor (FK)**
Ana	Carlos
Ana	Fernanda
Bruno	Carlos
Bruno	Fernanda
Carla	Carlos

Tabela 3.16: Tabela ALUNO_PROFESSOR, resultante da decomposição em FNBC.

Com a decomposição realizada, a dependência funcional `Professor` → `Curso_Extra` passa a ser representada em uma relação própria, na qual `Professor` atua como chave primária. Já a associação entre aluno e professor é armazenada separadamente, preservando as inscrições realizadas sem redundância indevida.

Dessa forma, elimina-se a violação anteriormente identificada, pois, em ambas as relações resultantes, todo determinante passa a ser chave.

3.4.2 Quarta Forma Normal (4FN)

A Quarta Forma Normal trata das dependências multivaloradas. Esse tipo de dependência ocorre quando uma mesma entidade possui dois ou mais conjuntos de valores independentes, associados a diferentes atributos, que são armazenados em múltiplas linhas da relação, mesmo não havendo qualquer relação semântica entre esses conjuntos de valores.

Importa destacar que a violação da Quarta Forma Normal não decorre da presença de atributos multivalorados no sentido clássico da Primeira Forma Normal. Na situação analisada, todos os valores são atômicos e a relação atende à 1FN. O problema surge em um nível mais profundo, em termos práticos, tem-se múltiplos conjuntos de valores independentes distribuídos em diferentes linhas, associados a atributos que não possuem relação entre si, mas que, devido à sua repetição na mesma relação, acabam sendo combinados, gerando associações artificiais e redundantes.

Essa estrutura dificulta a manutenção dos dados, uma vez que operações de inserção, atualização e exclusão passam a demandar múltiplas alterações correlacionadas, caracterizando anomalias semelhantes às observadas em formas normais inferiores.

Para resolver esse problema, torna-se necessária a decomposição da relação, separando-se esses conjuntos em relações distintas, cada uma representando um único fato multivalorado.

Assim, a Quarta Forma Normal busca garantir que cada relação represente apenas um único conjunto de associações multivaloradas, eliminando combinações artificiais e promovendo um modelo mais enxuto, consistente e alinhado aos princípios do modelo relacional.

Situação exemplo

Considere a seguinte relação que busca representar os perfis de atuação dos professores: `(Professor, Disciplina, Polo)`. Suponha que um professor possa lecionar várias disciplinas e atuar em diversos polos. Contudo, essas duas informações são independentes entre si, isto é, as disciplinas que o professor leciona não estão vinculadas aos polos em que ele atua.

Professor	Disciplina	Polo
Carlos	Banco de Dados	Campus Norte
Carlos	Banco de Dados	Campus Sul
Carlos	Redes	Campus Norte
Carlos	Redes	Campus Sul
Fernanda	Algoritmos	Campus Norte
Fernanda	Algoritmos	Campus Sul
Fernanda	Estruturas de Dados	Campus Norte
Fernanda	Estruturas de Dados	Campus Sul

Tabela 3.17: Relação com dependências multivaloradas.

Observa-se que, para cada professor, as disciplinas que ele leciona são combinadas com todos os polos em que atua, gerando associações artificiais entre esses dois conjuntos de valores. Como consequência, a relação passa a representar implicitamente que cada disciplina é ministrada em cada polo, o que não corresponde à realidade que se pretendia modelar.

Essa situação decorre do fato de que as informações `Disciplina` e `Polo` são independentes entre si, estando ambas associadas exclusivamente ao atributo `Professor`.

Na forma como a tabela foi estruturada, o banco de dados registra combinações entre `Disciplina` e `Polo` que não representam fatos reais, mas sim um cruzamento artificial entre valores independentes, introduzindo redundância na relação.

Tal redundância não decorre de uma exigência do domínio do problema, mas sim de uma inadequação na organização dos dados. Para eliminar essa inconsistência, impõe-se a decomposição da relação, separando-se os conjuntos multivalorados independentes em estruturas distintas.

Professor	Disciplina
Carlos	Banco de Dados
Carlos	Redes
Fernanda	Algoritmos
Fernanda	Estruturas de Dados

Tabela 3.18: Relação PROFESSOR_DISCIPLINA.

Professor	Polo
Carlos	Campus Norte
Carlos	Campus Sul
Fernanda	Campus Norte
Fernanda	Campus Sul

Tabela 3.19: Relação PROFESSOR_POLO.

Com a decomposição realizada, eliminam-se as dependências multivaloradas, garantindo que cada relação represente um único conjunto de associações.

3.4.3 Quinta Forma Normal (5FN ou PJNF)

Até este ponto, considera-se implicitamente que o processo de normalização se dá por meio da decomposição de uma relação em duas novas relações, de forma que a junção dessas partes seja suficiente para reconstruir os dados originais sem perda de informação. Essa abordagem é suficiente para atender às exigências até a Quarta Forma Normal.

Entretanto, existem situações em que essa premissa não se sustenta. Há relações que não podem ser corretamente decompostas em apenas duas novas relações sem perda de informação, sendo necessária a sua decomposição em três ou mais relações para que a reconstrução por junção preserve integralmente os dados originais.

Nesses casos, a decomposição adequada depende da combinação simultânea de múltiplas relações, o que evidencia a presença de dependências de junção mais complexas, não capturadas por decomposições binárias. Esse fenômeno reforça a necessidade da Quinta Forma Normal.

A Quinta Forma Normal, também conhecida como Forma Normal de Projeção-Junção (PJNF), trata dessas situações em que uma relação pode ser decomposta em várias outras menores, de modo que apenas a junção de todas elas permite reconstruir corretamente os dados originais, sem gerar tuplas inválidas.

Esse tipo de problema está relacionado às chamadas **dependências de junção**, que ocorrem quando a associação entre três ou mais atributos depende da combinação conjunta entre eles, não podendo ser corretamente representada por decomposições parciais.

Em termos práticos, a 5FN busca evitar a existência de combinações inválidas de dados que podem surgir quando uma relação é reconstruída a partir de partes que, isoladamente, não capturam completamente a regra de negócio.

Situação exemplo

Considere a seguinte relação que busca representar a participação de professores em atividades acadêmicas: `(Professor, Disciplina, Projeto)`

Professor	Disciplina	Projeto
Carlos	Banco de Dados	Pesquisa IA
Carlos	Redes	Pesquisa Redes
Fernanda	Algoritmos	Pesquisa IA

Tabela 3.20: Relação envolvendo professor, disciplina e projeto.

Suponha que no domínio que estamos modelando tenhamos as seguintes regras:

- R1 - um professor pode lecionar determinadas disciplinas;

- R2 - um professor pode participar de determinados projetos;
- R3 - determinadas disciplinas podem estar associadas a determinados projetos.
- R4 - nem toda combinação entre `Professor`, `Disciplina` e `Projeto` é válida, apenas algumas combinações específicas representam situações reais.

Observa-se que, na relação apresentada, não há dependências multivaloradas independentes, uma vez que as disciplinas e os projetos estão diretamente associados entre si. Dessa forma, a relação atende aos requisitos da Quarta Forma Normal.

Contudo ainda há redundância de dados e, devido a R4, não é possível decompor essa relação em apenas duas tabelas sem gerar combinações inválidas. Caso fossem criadas, por exemplo, as relações `(Professor, Disciplina)` e `(Professor, Projeto)`, a junção dessas relações resultaria em novas combinações que não existiam originalmente, associando disciplinas e projetos de forma indevida, conforme demonstrado abaixo:

Professor	**Disciplina**
Carlos	Banco de Dados
Carlos	Redes
Fernanda	Algoritmos

Professor	**Projeto**
Carlos	Pesquisa IA
Carlos	Pesquisa Redes
Fernanda	Pesquisa IA

Ao realizar a junção natural entre essas duas relações, obtém-se o seguinte resultado:

Professor	**Disciplina**	**Projeto**
Carlos	Banco de Dados	Pesquisa IA
Carlos	Banco de Dados	Pesquisa Redes
Carlos	Redes	Pesquisa IA
Carlos	Redes	Pesquisa Redes
Fernanda	Algoritmos	Pesquisa IA

Tabela 3.21: Junção das relações `(Professor, Disciplina)` e `(Professor, Projeto)`, com destaque para combinações inválidas.

Perceba que surgem combinações espúrias, como `(Carlos, Banco de Dados, Pesquisa Redes)` e `(Carlos, Redes, Pesquisa IA)`, que não constavam na relação original e não representam situações reais do domínio modelado.

Isso ocorre porque a validade dos dados depende da associação simultânea entre os três atributos, e não apenas de pares isolados. Para preservar corretamente os dados, torna-se necessária a decomposição em três relações distintas:

Professor	Disciplina
Carlos	Banco de Dados
Carlos	Redes
Fernanda	Algoritmos

Professor	Projeto
Carlos	Pesquisa IA
Carlos	Pesquisa Redes
Fernanda	Pesquisa IA

Disciplina	Projeto
Banco de Dados	Pesquisa IA
Redes	Pesquisa Redes
Algoritmos	Pesquisa IA

A junção natural dessas três relações permite reconstruir exatamente os dados originais, sem gerar combinações inválidas. Dessa forma, elimina-se a dependência de junção presente na relação inicial.

3.5 Resumo Prático de Normalização

Nesta seção, realizaremos a normalização de uma planilha de empréstimos de uma biblioteca universitária até a FNBC, pois, considera-se que a FNBC seja suficiente para a maioria dos cenários práticos, embora existam situações em que ainda podem persistir redundâncias não resolvidas por ela.

Problema: A Ficha de Empréstimo de Livros (FNN), contendo as seguintes informações: matricula do aluno, nome do aluno, curso, data de emprestimo, data prevista de devolucao, ISBN do livro, titulo do livro, codigo da editora, nome da editora, categoria do livro.

Matricula	Nome	Curso	Data_Emp	Data_Dev	ISBN	Título	Cod_Ed	Editora	Categoria
101	João	Eng. de Software	01/03	10/03	L1	Banco de Dados	E1	Atlas	TI
					L2	Redes	E2	Pearson	TI
102	Maria	Direito	02/03	12/03	L3	Direito Civil	E3	Saraiva	Jurídico

Tabela 3.22: Tabela em forma não normalizada (FNN) com grupos repetitivos.

Passo 1: Entender Atributos e Grupo Repetitivo.

Em um mesmo empréstimo, o estudante pode locar vários livros. Assim, os atributos `ISBN_Livro`, `Titulo_Livro`, `Codigo_Editora`, `Nome_Editora` e `Categoria` se repetem para cada item, caracterizando grupos repetitivos.

Para adequar à 1FN, cada linha deve representar apenas um único livro por empréstimo, eliminando-se os grupos repetitivos. Para isso, a relação original é reorganizada em

duas estruturas:

Matricula_Aluno	Nome_Aluno	Curso	**Data_Emprestimo**	Data_Devolucao_Prevista
101	João	Eng. de Software	01/03	10/03
102	Maria	Direito	02/03	12/03

Tabela 3.23: Relação EMPRESTIMO (estrutura intermediária).

Matricula_Aluno	**Data_Emprestimo**	**ISBN_Livro**	Titulo_Livro	Codigo_Editora	Nome_Editora	Categoria
101	01/03	L1	Banco de Dados	E1	Atlas	TI
101	01/03	L2	Redes	E2	Pearson	TI
102	02/03	L3	Direito Civil	E3	Saraiva	Jurídico

Tabela 3.24: Relação ITEM_EMPRESTIMO.

Com essa reorganização, os grupos repetitivos são eliminados, pois cada linha da relação `ITEM_EMPRESTIMO` passa a representar um único livro associado a um empréstimo específico. Dessa forma, a estrutura passa a atender aos requisitos da Primeira Forma Normal. Contudo, ainda permanecem dependências parciais na estrutura obtida.

Passo 2: Aplicação da 2FN

Na relação `EMPRESTIMO`, os atributos `Nome_Aluno` e `Curso` dependem exclusivamente de `Matricula_Aluno`, não sendo atributos do evento de empréstimo propriamente dito.

Na relação `ITEM_EMPRESTIMO`, os atributos `Titulo_Livro`, `Codigo_Editora`, `Nome_Editora` e `Categoria` dependem apenas de `ISBN_Livro`.

Dessa forma, para adequação à Segunda Forma Normal, torna-se necessária nova decomposição, separando-se os atributos que dependem apenas de parte da chave composta.

Matricula_Aluno	Nome_Aluno	Curso
101	João	Eng. de Software
102	Maria	Direito

Tabela 3.25: Tabela ALUNO.

ISBN_Livro	Titulo_Livro	Codigo_Editora	Nome_Editora	Categoria
L1	Banco de Dados	E1	Atlas	TI
L2	Redes	E2	Pearson	TI
L3	Direito Civil	E3	Saraiva	Jurídico

Tabela 3.26: Tabela LIVRO.

Matricula_Aluno	**Data_Emprestimo**	Data_Devolucao_Prevista
101	01/03	10/03
102	02/03	12/03

Tabela 3.27: Tabela EMPRESTIMO.

Matricula_Aluno	**Data_Emprestimo**	**ISBN_Livro**
101	01/03	L1
101	01/03	L2
102	02/03	L3

Tabela 3.28: Tabela ITEM_EMPRESTIMO.

Com essa reorganização, eliminam-se as dependências parciais, atendendo aos requisitos da Segunda Forma Normal.

Passo 3: Aplicação da 3FN

Observando a tabela `LIVRO` resultante do passo anterior, verifica-se a existência de uma dependência transitiva: o atributo `Nome_Editora` depende de `Codigo_Editora`, que, por sua vez, depende da chave primária `ISBN_Livro`. Para atender à Terceira Forma Normal, atributos não-chave que dependem de outros atributos não-chave devem ser movidos para uma nova relação.

Vale ressaltar que o atributo `Categoria` depende diretamente da chave primária `ISBN_Livro` e não determina outros atributos. Portanto, sua permanência não viola a 3FN, sendo mantido na tabela original. Ao realizar a extração da editora, é fundamental manter a chave estrangeira na tabela de origem para não perder o relacionamento.

Com essa reorganização, eliminam-se as dependências transitivas, promovendo maior independência entre as entidades e preservando a integridade referencial através das chaves estrangeiras.

Abaixo, apresenta-se o modelo de dados completo e consolidado após a aplicação da 3FN, refletindo todas as tabelas e relacionamentos extraídos até o momento:

Matricula_Aluno	Nome_Aluno	Curso
101	João	Eng. de Software
102	Maria	Direito

Tabela 3.29: Tabela ALUNO.

Codigo_Editora	Nome_Editora
E1	Atlas
E2	Pearson
E3	Saraiva

Tabela 3.30: Tabela EDITORA.

ISBN_Livro	Titulo_Livro	Codigo_Editora (FK)	Categoria
L1	Banco de Dados	E1	TI
L2	Redes	E2	TI
L3	Direito Civil	E3	Jurídico

Tabela 3.31: Tabela LIVRO após a adequação à 3FN.

Matricula_Aluno	**Data_Emprestimo**	Data_Devolucao_Prevista
101	01/03	10/03
102	02/03	12/03

Tabela 3.32: Tabela EMPRESTIMO.

Matricula_Aluno	**Data_Emprestimo**	**ISBN_Livro**
101	01/03	L1
101	01/03	L2
102	02/03	L3

Tabela 3.33: Tabela ITEM_EMPRESTIMO.

Passo 4: Aplicação da FNBC Considerando a estrutura já obtida, observa-se que as relações `ALUNO`, `EMPRESTIMO`, `ITEM_EMPRESTIMO`, `LIVRO`, `EDITORA` e `CATEGORIA` não apresentam dependências funcionais em que o determinante deixe de ser chave.

Assim, não se identifica, no modelo construído, violação à Forma Normal de Boyce-Codd. Dessa forma, conclui-se que a estrutura final já atende aos requisitos da FNBC.

Com isso, conclui-se a normalização do modelo, garantindo a eliminação de redundâncias e a adequada organização das informações.

Passo Extra: recomendação prática de mercado com chaves artificiais (IDs) e tabelas de domínio

Embora o modelo construído atenda integralmente aos requisitos teóricos até a FNBC, na prática de modelagem de bancos de dados relacionais e no desenvolvimento de software, adota-se comumente o uso de **chaves primárias artificiais** (IDs numéricos e autoincrementais) para entidades cujos atributos naturais podem sofrer alterações.

Essa prática visa identificar cada registro de forma única e imutável, simplificando os relacionamentos (*Foreign Keys*) e protegendo o modelo contra mudanças nas regras de negócio externas (como a alteração no formato de um ISBN). Contudo, atributos naturais que já possuem controle interno estrito pelo próprio sistema ou instituição, como a `Matricula_Aluno`, atuam de forma excelente como chaves primárias, dispensando a criação de IDs redundantes.

Além disso, embora a extração do atributo `Categoria` não seja uma exigência da 3FN ou FNBC (pois este depende exclusivamente da chave do livro), separá-lo em uma tabela própria (conhecida como Tabela de Domínio ou *Lookup Table*) é altamente recomendado. Isso garante a integridade de domínio, evitando anomalias de inserção causadas por digitação inconsistente (ex: "TI", "T.I."e "Tecnologia") e centralizando as opções disponíveis

no sistema.

Abaixo, apresenta-se o modelo final do banco de dados da biblioteca universitária, aplicando as chaves artificiais onde necessário, mantendo a chave natural controlada para os alunos, e isolando a categoria:

Matricula_Aluno	Nome_Aluno	Curso
101	João	Eng. de Software
102	Maria	Direito

Tabela 3.34: Tabela ALUNO mantendo a Matrícula como chave primária natural controlada.

ID_Categoria	Nome_Categoria
1	TI
2	Jurídico

Tabela 3.35: Tabela CATEGORIA (Tabela de Domínio).

ID_Editora	Codigo_Editora	Nome_Editora
1	E1	Atlas
2	E2	Pearson
3	E3	Saraiva

Tabela 3.36: Tabela EDITORA.

ID_Livro	ISBN_Livro	Titulo_Livro	ID_Editora (FK)	ID_Categoria (FK)
1	L1	Banco de Dados	1	1
2	L2	Redes	2	1
3	L3	Direito Civil	3	2

Tabela 3.37: Tabela LIVRO referenciando Editora e Categoria via IDs.

ID_Emprestimo	Matricula_Aluno (FK)	Data_Emprestimo	Data_Devolucao_Prevista
1	101	01/03	10/03
2	102	02/03	12/03

Tabela 3.38: Tabela EMPRESTIMO com chave artificial única e referenciando a Matrícula.

ID_Emprestimo (FK)	ID_Livro (FK)
1	1
1	2
2	3

Tabela 3.39: Tabela associativa ITEM_EMPRESTIMO simplificada pelas chaves artificiais e de empréstimo.

Com essa estrutura final, o modelo ganha resiliência, organização e obedece rigorosamente aos padrões de qualidade para o desenvolvimento de sistemas comerciais modernos.

3.6 Exercícios Propostos

Abaixo temos uma visão geral de informações mantidas, tradicionalmente em Excel, pela coordenação de estágios de um instituto universitário. Uma mesma empresa fornecedora de estágio pode ofertar vagas com orientadores distintos.

Planilha de Estágio: (CNPJ_Empresa, Nome_Empresa, Endereco_Empresa, Matrícula_Aluno, Nome_Aluno, Curso_Aluno, Id_Orientador, Nome_Orientador, Titulacao_Orientador, Data_Inicio_Estagio, Horas_Mensais)

Questão 1: Descreva, com um exemplo em papel, quais tipos de Anomalia de Atualização e Anomalia de Exclusão poderiam ocorrer exatamente devido à estrutura acima, caso a empresa precisasse mudar de endereço em São Paulo.

Questão 2: Identifique os Grupos Repetitivos e liste qual seria a chave candidata composta ideal para normalizar o formulário diretamente para a sua Primeira Forma Normal (1FN). Apresente o resultado.

Questão 3: Conduza os dados gerados da 1FN aos moldes da Segunda Forma Normal (2FN). Quantas e quais tabelas resultaram deste processo de eliminação de dependências funcionais parciais? Elenque as chaves de tabela (PK e FK).

Questão 4: Prossiga com as tabelas geradas na Questão 3 e promova o projeto à sua Terceira Forma Normal (3FN), removendo quaisquer dependências transitivas. Especifique com clareza quais tabelas extras surgiram para alocar a entidade responsável. O seu Diagrama Final representará de fato uma modelagem íntegra e sem distorções operacionais.

Conclusão: A proficiência no uso da normalização de dados separará engenheiros medianos de profissionais capazes de escalar, modernizar ou remodelar enormes cargas de responsabilidade estrutural. Lembre-se, normalizar primeiro e desnormalizar na arquitetura, sempre quando necessário.

Referências

ANGELOTTI, Elaini Simoni. **Banco de dados**. Curitiba: Editora do Livro Técnico, 2010. p. 120. ISBN 978-85-63687-02-9.

CODD, E. F. Recent Investigations into Relational Data Base Systems. *In:* PROCEEDINGS of the IFIP Congress 1974. [*S. l.: s. n.*], 1974. p. 1017–1021.

CODD, Edgar F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM**, Association for Computing Machinery, v. 13, n. 6, p. 377–387, 1970. DOI: `10.1145/362384.362685`.

CODD, Edgar F. **Further Normalization of the Data Base Relational Model**. San Jose, California, 1971.

DATE, C. J. **Introdução a Sistemas de Banco de Dados**. Tradução: Daniel Vieira. Rio de Janeiro: Elsevier, 2003. Tradução de: Introduction to Database Systems, 8th ed. ISBN 978-85-352-8445-4.

HEATH, I. J. Unacceptable File Operations in a Relational Database. *In:* PROCEEDINGS of the 1971 ACM SIGFIDET Workshop on Data Description, Access, and Control. San Diego, California: [*s. n.*], nov. 1971.

KENT, William. A Simple Guide to Five Normal Forms in Relational Database Theory. **Communications of the ACM**, v. 26, n. 2, p. 120–125, fev. 1983. DOI: `10.1145/358061.358064`.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados**. Tradução: Célia Taniawake. 3. ed. Porto Alegre: AMGH, 2011. Recurso eletrônico. Editado também como livro impresso em 2008. ISBN 978-85-63308-77-1.